PRODUCT LEADERSHIP SERIES · 2026 EDITION

The Product 360 Handbook

Building, Running and Governing Products in the AI Era

A field guide and operating system for product leaders, 2026 edition.

Curated by Arunabh Bhattacharjee

FRONT MATTER

Preface: How to Use This Book

Who this book is for, how it is organized, and how to turn it into your own Product 360.

5 min read

This book is for the person who just got handed a product and told to own it. Not a feature, not a backlog: the whole thing. Its customers, its strategy, its numbers, its launches, its operations, and eventually its retirement.

That job has a name in most companies (Senior PM, Principal PM, Group PM, product lead) and almost no manual. You learn it in pieces: discovery from one book, OKRs from another, launch tiers from a product marketing friend, end-of-life policy from a lawyer, AI evals from a podcast. Then you spend years stitching the pieces into something you can run.

The Product 360 Handbook is that stitching, done once, in one place.

What “Product 360” means

A Product 360 is the complete, living picture of a product, and the set of artifacts and rituals that keep that picture true. It covers nine dimensions: customer, strategy, business, build, measure, go-to-market, operate, lifecycle and leadership. AI runs through all nine, because in 2026 it changes both what you build and how you do the work.

Chapter 00 defines the model in detail. The short version: if you can open your Product 360 and answer "who is this for, why will we win, how do we make money, what are we building next, how do we know it works, how does it reach customers, how do we run it, where is it in its life, and who needs to be aligned," you own the product. If you can't, the gaps tell you what to build first.

Who this book is for

The primary reader is a Senior to Principal product manager in a B2B, enterprise or platform business. Newer PMs will find **On-ramp** boxes that cover the basics a chapter assumes. Directors and VPs will find **For product leaders** boxes on portfolio, org and governance questions.

The examples lean enterprise and platform, because that is where the Product 360 problem is hardest: long sales cycles, many stakeholders, partner channels, compliance reviews and products that live for a decade. Consumer examples show up where a concept is clearer in consumer.

How the book is organized

The book starts with the core craft and widens from there.

- **Part I, Product Management**, covers the core loop in the order you practice it: hypothesis and personas, vision and goals, strategy and roadmap, prototyping, analytics and iteration, engineering and execution.
- **Part II, Technical Product Management**, adds the technical and quantitative depth: how products fail, data-driven strategy, analytics for retention and lifetime value, consumer insight, delivery methods and value-based pricing.
- **Part III, Business and Market Strategy**, covers market sizing, business cases, platforms and network effects, pricing and packaging, and defensibility.

- **Part IV, Data-Driven Decisions in the AI Era**, teaches what to ask, which data answers it and which reports a product leader should own. It teaches no query syntax, on purpose: in 2026 the harness writes the query, and your job is to ask the right question and check the answer.
- **Part V, Innovation**, covers how established companies keep inventing.
- **Part VI, Go-to-Market**, covers positioning, the offer, launches, announcements, events, enablement and customer success.
- **Part VII, Product Operations and Lifecycle Management**, covers the operating cadence, governance, business reviews, stage gates and end-of-life.
- **Part VIII, AI-Era Product Management**, goes deep on building AI products, doing the PM job with AI, and responsible AI and regulation.
- **Part IX, Influence and Leadership**, covers writing, presenting and stakeholder leadership.
- **The Capstone** turns the whole book into a playbook for building your own Product 360, including a set of grilling prompts you can run with Claude or any AI harness.

The book is self-contained. Where a topic shows up in more than one part (evals, pricing, AI risk), each chapter covers what its reader needs and points to the other chapters that cover it.

How every chapter works

Each chapter follows the same shape, so you can use it as a reference as well as read it straight through:

1. **Why this matters**: the job the chapter helps you do.
2. **Core concepts and frameworks**: the strongest thinking on the topic, attributed to the people who developed it.
3. **How to apply it**: a step-by-step method.
4. **The AI lens**: what changes when the product is AI-powered, and what AI changes about doing the work.

5. **The Beacon case**: the running example, worked through.
6. **Product 360 artifacts**: the templates the chapter produces.
7. **Prompts**: copy-ready prompts for the chapter's tasks.
8. **Anti-patterns**: how this usually goes wrong.
9. **Checklist**: what "done" looks like. Tick the boxes; your browser remembers them.
10. **Key takeaways**: the points to carry into your own Product 360.

The running case: Beacon

Every chapter works through the same fictional product so you can watch a Product 360 come together. **Beacon** is an AI agent platform from a fictional company, Northlight Systems, that helps IT operations and SRE teams detect, diagnose and resolve incidents. It started as an alert-correlation feature, became a standalone product, and in 2026 is moving into agentic diagnosis and supervised remediation. You play its newly appointed Principal PM.

Beacon was chosen because it has most of the hard problems a product leader faces: multiple buyer personas, a platform-versus-standalone strategy question, AI cost eating gross margin, a pricing model in transition, a big launch at a customer event, a legacy product to sunset and EU customers asking for AI Act documentation. Beacon, Northlight and every number attached to them are invented.

Conventions

NOTE **How to read the boxes**

Key idea boxes hold the one thing to remember. **The AI lens** boxes cover AI-specific implications. **Beacon case** boxes carry the running example. **Anti-pattern** boxes describe common failures. **On-ramp** and **For product leaders** boxes adjust the depth up or down.

Prompts are model-agnostic. They work in Claude, ChatGPT, Gemini, Copilot or an internal harness. Anything in `[square brackets]` is a placeholder for your own context. Every prompt says what to paste in and how to check what comes back, because an AI answer you didn't check is a guess with good formatting.

Templates are plain Markdown, so they paste cleanly into Confluence, Notion, Google Docs or a GitHub repo. Every template is also collected in Appendix A, with a download link.

Tools and vendors are named as categories first, with examples labeled as of late 2026. Tools change faster than books do.

How to use this book to build your Product 360

If you are new to a product area, don't read the book front to back on day one. Do this instead:

1. Read Chapter 00 to get the model, the artifact stack and the cadence.
2. Score your product with the maturity model in Appendix C. Find the two weakest dimensions.
3. Jump to the chapters for those dimensions and build their artifacts first.
4. Use the Capstone's 30/60/90 plan and grilling prompts to fill in the rest with your AI harness.
5. Run the operating cadence from Chapter 28, so the 360 stays true after you've built it.

A note on independence

This handbook is a companion text for the topics in the eCornell Product Management 360 certificate, and Appendix E maps each chapter to a course. It is independent and is not affiliated with or endorsed by Cornell University or eCornell. Nothing in it is legal, financial or regulatory advice; the regulation chapters describe the landscape so you know which questions to bring to your counsel.

FRONT MATTER

00

The Product 360 Framework

The model, the artifact stack and the operating cadence the whole book hangs on.

26 min read · 3 templates · 4 prompts

Owning a product means you can answer nine questions about it at any moment, show the artifact that holds each answer, and say when that artifact was last made true. The Product 360 is the model for those nine questions, the stack of artifacts that answers them, and the operating cadence that keeps the answers current.

Why this matters

Most product managers inherit a product the same way: a handoff document, a Jira project, a roadmap slide that is two quarters stale, a Slack channel and a list of people to meet. Somewhere in there is the strategy. Some of it lives in a deck from last year's planning cycle, some in the head of the PM who just left, and some nowhere at all.

You can run a product like that for a while. Then the questions start arriving faster than you can reconstruct the answers. Sales asks why a competitor's feature isn't on the roadmap. Finance asks why gross margin dropped four points. A customer asks when the legacy version reaches end of support. An executive asks for the three-year view by Friday. Each question gets answered from scratch, slightly differently each time, and the product drifts toward whoever asked most recently.

A Product 360 fixes this by making the product legible: to you, to your team, to your executives and, in 2026, to the AI tools you work with. Rather than one big document, it's a small, connected set of artifacts, each owning one part of the picture, plus a rhythm for updating them. When it works, you stop rebuilding context and start using it.

This chapter defines the model. Every other chapter in the book goes deep on one part of it, and the Capstone shows you how to build your own.

What a product manager is for

Before defining the 360, be precise about the job it serves, because "product manager" covers wildly different roles across companies.

The four risks

Marty Cagan's framing from *Inspired* is still the cleanest starting point. Every product idea carries four risks:

- **Value risk**: will customers buy it, or choose to use it?
- **Usability risk**: can users figure out how to use it?
- **Feasibility risk**: can engineering build it with the time, skills and technology available?
- **Business viability risk**: does it work for the rest of the business: sales, marketing, finance, legal, support, partners?

In Cagan's model, the product manager is accountable for value and viability, the designer for usability, and the tech lead for feasibility. The team owns all four together. That split matters for the 360: the artifacts a PM must own are the ones that answer value and viability questions, and the artifacts the PM must be able to read and challenge are the ones that answer usability and feasibility.

Cagan's later books push the model up a level. *Empowered* argues that strong product teams are given problems to solve, not features to build. *Transformed* describes the "product operating model": how a company changes the way it decides what to build (discovery before delivery), how it builds (small, frequent, instrumented releases) and how it chooses problems (strategy and insight rather than a stakeholder wish list). The Product 360 is, among other things, the set of artifacts that makes a product operating model visible.

Three intersections

Martin Eriksson's well-known diagram places product management at the intersection of business, technology and user experience. It's a useful reminder that a PM who lives entirely in one circle stops doing the job. A PM who only talks to engineers becomes a project manager. One who only talks to sales becomes a feature broker. One who only talks to designers loses the business case.

The 360 enforces the intersection structurally. You can't keep it current without regularly touching all three circles.

Ken Norton adds the part of the job that no diagram captures: a PM leads by influence, without direct reports, and acts as the advocate "for whoever isn't currently in the room." Sometimes that's the customer. Sometimes it's engineering, sales or finance. The 360 is how the people who aren't in the room still get represented, because their part of the picture is written down.

How AI is changing the role

The PM role is shifting, and the people who defined it are still arguing about how much. In early 2025 Cagan sketched a future in which a typical product team shrinks from about eight people to about three (a PM, a designer and an engineer), with delivery largely automated and discovery becoming the main work. By mid-2026 he had tempered that. In "The AI Productivity Paradox" he argued that teams using AI to speed up an output-focused process ship more without getting better outcomes, while strong product organizations pull further ahead. A month later he wrote that his earlier optimism about AI turning many more people into strong "product creators"

had been too high, because the hard part was never the code but knowing what should exist.

Practitioner data points the same way. A late-2025 survey of about 1,750 tech workers by Lenny Rachitsky and Noam Segal found PMs using AI most for writing PRDs and building prototypes, with nearly two-thirds saving four or more hours a week, and far fewer using it for upstream thinking like research or roadmap choices. Norton's twenty-year retrospective on his own essay makes a similar argument: the human parts of the job (judgment, influence, advocacy) are the least likely to be automated.

The practical reading for this book: AI makes artifacts cheaper to produce, so producing them is no longer the hard part. The job is knowing which ones matter, making the choices inside them and keeping them true. The Product 360 is built around that.

Three levels of product work

Shreyas Doshi distinguishes three levels at which product work happens: **impact** (are we working on the things that will move the business?), **execution** (are we doing those things well?) and **optics** (does it look like we're doing them well?). Teams get into trouble when they argue at different levels. An executive asking an impact question ("should we be in this market?") gets an execution answer ("we're on track for the March release").

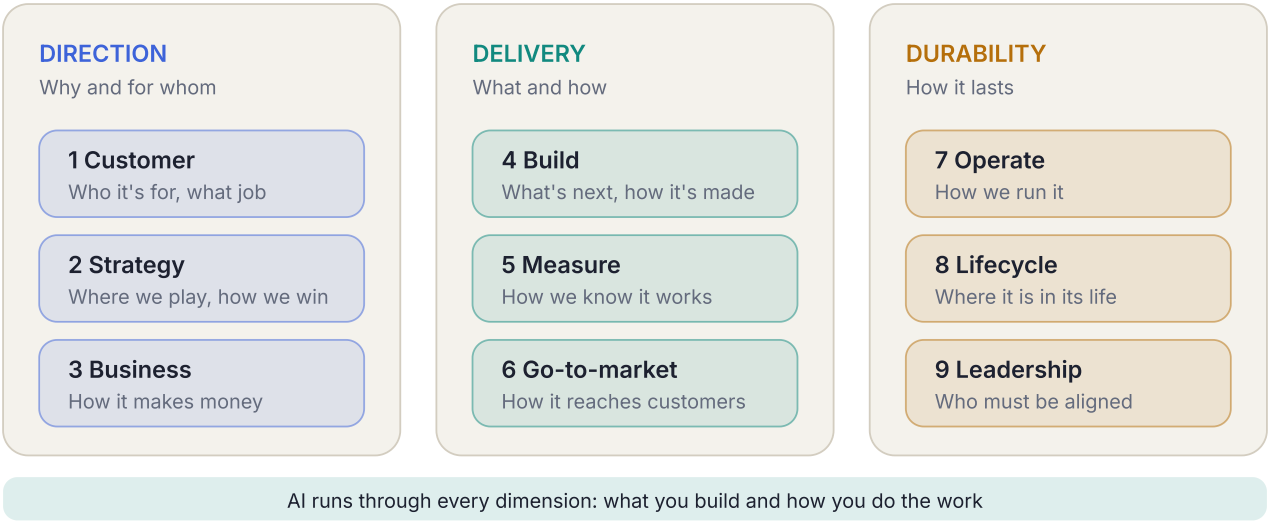
The Product 360 is organized to keep those levels separate. Strategy artifacts answer impact questions. Plans, specs and metrics answer execution questions. Reviews and communication artifacts handle optics honestly: they're how you show the work without performing it.

ON-RAMP **If you're new to product management**

A PM's job is to decide, with the team, which problems are worth solving and to make sure the solution creates value for customers and the business. You don't need to be the smartest person on any single dimension. You need to be the person who sees all of them at once and makes the tradeoffs explicit. That "seeing all of them at once" is what a Product 360 is for.

The nine dimensions of the Product 360

The Product 360 divides the product into nine dimensions. Each one answers a question someone will eventually ask you, and each maps to a part of this book.



The nine dimensions, grouped by the kind of question they answer. AI is not a tenth dimension; it changes all nine.

Dimension	The question it answers	Core artifacts	Where in this book
1. Customer	Who is this for, and what job are they hiring it to do?	Personas, JTBD statements, journey maps, problem hypothesis, discovery log	Ch. 01, 10
2. Strategy	Where do we play, how do we win, and what will we say no to?	Vision, strategy one-pager, competitive landscape, platform map	Ch. 02, 03, 13, 14, 16
3. Business	How does this make money, and is the money worth it?	Business case, pricing and packaging, unit economics model	Ch. 12, 13, 15
4. Build	What are we building next, and how does it get made?	Roadmap, PRDs and PRFAQs, backlog, release plan	Ch. 03, 04, 06, 11
5. Measure	How do we know it’s working?	North Star and metric tree, OKRs, dashboards, experiment readouts, evals	Ch. 02, 05, 09, 17 to 19
6. Go-to-market	How does it reach, convince and keep customers?	Positioning, messaging, launch plans, enablement, announcements	Ch. 24 to 27
7. Operate	How do we run it week to week?	Operating cadence, business reviews, decision log, risk register	Ch. 28, 29

Dimension	The question it answers	Core artifacts	Where in this book
8. Lifecycle	Where is it in its life, and what happens next?	Lifecycle stage assessment, stage-gate criteria, sunset plans	Ch. 30
9. Leadership	Who needs to be aligned, and are they?	Stakeholder map, DACI, communication plan, team charter	Ch. 34 to 36

The nine group into three families. **Direction** (customer, strategy, business) answers why and for whom. **Delivery** (build, measure, go-to-market) answers what and how. **Durability** (operate, lifecycle, leadership) answers how the product keeps working as a business over years, which is the part PM training often skips.

KEY IDEA **The test of a Product 360**

Pick any of the nine questions. If you can open one artifact, show the answer and say when it was last made true, that dimension is covered. If the answer lives in a meeting, a person or a slide from last year, it isn't.

Why AI is not a tenth dimension

It would be easy to add "AI" as a tenth box. That would be a mistake, because in 2026 AI changes every one of the nine, in two different ways.

AI in the product. When the product itself is AI-powered, each dimension gets new questions. Customer: how much do users trust an answer they can't verify? Business: what does each inference cost, and does pricing cover it? Build: what's the eval suite, and which model are we on? Measure: how do we measure quality when the output isn't deterministic? Operate: who responds when the model starts producing worse answers after a provider update? Lifecycle: what happens to customers when the underlying model is deprecated? Chapter 31 covers these in depth, and every chapter has an AI lens section for its own dimension.

AI in the work. Separately, AI changes how you build and maintain the 360 itself. A harness like Claude, with connectors into your docs, tickets, CRM and analytics, can draft an artifact from raw material, check one artifact against another, and answer questions across the whole set. That only works if the 360 exists in a form the harness can read. Chapter 32 covers this, and the Capstone gives you the prompts.

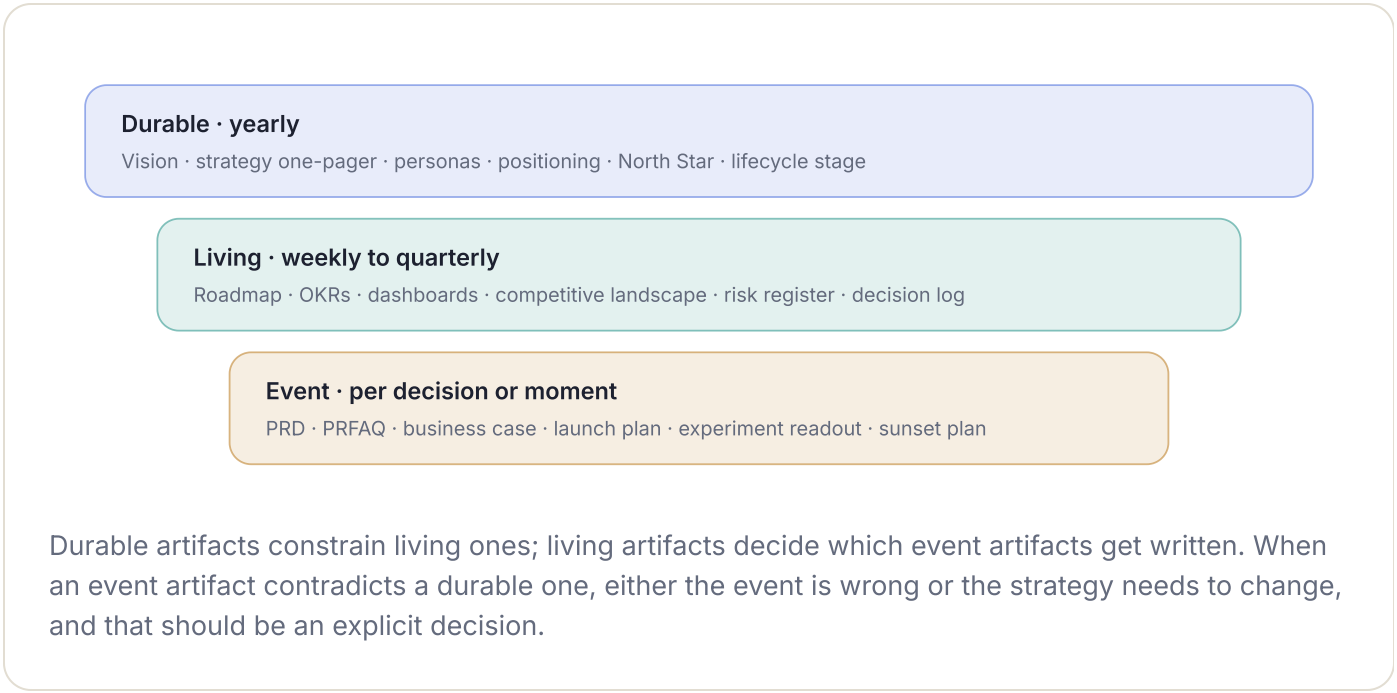
The artifact stack

A Product 360 is built from artifacts. Not every document is an artifact. An artifact, in this book, is a document with an owner, a purpose, an audience, and an update rhythm. If any of those four is missing, the document will rot.

Three kinds of artifact

Artifacts change at different speeds, and treating them all the same is how teams end up with either a stale strategy or a strategy that changes every week.

- **Durable artifacts** change rarely, usually once a year or at a major strategic shift. Vision, strategy one-pager, personas, positioning. If these change monthly, you don't have a strategy; you have a mood.
- **Living artifacts** change on a cadence: weekly, monthly or quarterly. Roadmap, metrics dashboard, OKRs, risk register, decision log, competitive landscape. If these go a quarter without an update, they're lying to someone.
- **Event artifacts** are created for a specific decision or moment and then archived. PRDs, PRFAQs, launch plans, experiment readouts, business cases, post-launch reviews, sunset plans. They're written once, used intensely, and then become the history that explains why the product looks the way it does.



Durable artifacts constrain living ones; living artifacts decide which event artifacts get written. When an event artifact contradicts a durable one, either the event is wrong or the strategy needs to change, and that should be an explicit decision.

The stack has a direction. Durable artifacts constrain living ones: the roadmap should be explainable from the strategy. Living artifacts decide which event artifacts get written: a PRD exists because something made it onto the roadmap. When an event artifact contradicts a durable one (a PRD for a segment the strategy says you don't serve), you've found either a bad PRD or a strategy that needs revisiting. Both are worth knowing, and the stack makes the conflict visible.

The minimum viable 360

The full artifact list in this book runs to about a hundred templates. You won't maintain a hundred documents, and you shouldn't try. Most products need a spine of about twelve, with everything else created when an event calls for it.

#	Artifact	Kind	Dimension
1	Product 360 index (the home page that links everything)	Living	All
2	Problem hypothesis and personas	Durable	Customer
3	Vision and strategy one-pager	Durable	Strategy
4	Competitive landscape	Living, quarterly	Strategy
5	Business model and unit economics summary	Living, quarterly	Business
6	Outcome roadmap (now / next / later)	Living, monthly	Build
7	North Star and metric tree, with the dashboard	Living, weekly	Measure
8	OKRs	Living, quarterly	Measure
9	Positioning and messaging	Durable	Go-to-market
10	Operating cadence and decision log	Living, weekly	Operate
11	Lifecycle stage assessment	Durable, reviewed yearly	Lifecycle
12	Stakeholder map and DACI	Living, quarterly	Leadership

If you're building a 360 from nothing, build these twelve first, in roughly this order. The Capstone walks through it in a 30/60/90-day plan.

ANTI-PATTERN **Template theater**

The fastest way to kill a Product 360 is to create all hundred templates on day one, fill them with plausible text, and never touch them again. A half-empty 360 that's true is worth more than a complete one that's fiction. Build an artifact when a question demands it, and keep it only if someone uses it.

Where the 360 lives

The 360 needs one home: a single page (the **Product 360 index**) that links every artifact, names its owner and shows when it was last updated. That page can live in Confluence, Notion, Google Drive, a SharePoint site or a Git repository. The tool matters less than three properties:

1. **One entry point.** Anyone new to the product starts at the index.
2. **Visible freshness.** Every artifact shows its owner and last-reviewed date. Stale artifacts are flagged, not hidden.
3. **Readable by machines.** In 2026 this one matters more than it used to. If your artifacts are in formats an AI harness can read through a connector (pages, docs, Markdown files), the harness can answer questions across the whole 360. If they're screenshots of slides, it can't.

The operating cadence

Artifacts don't keep themselves true. People do, on a rhythm. The operating cadence is the calendar that says which artifact gets reviewed when, by whom, and what decision the review produces.

The cadence below is this book's synthesis, and it borrows from several established practices. Amazon's weekly business review (WBR), as Colin Bryar and Bill Carr describe it in *Working Backwards*, has metric owners walk a fixed set of metrics every week and explain the variances. The quarterly business review (QBR) is standard in

enterprise software. Annual planning cycles like Amazon's OP1 and OP2 have teams write next year's plan as a narrative in the fall and revise it into the plan of record after the holiday quarter. Melissa Perri and Denise Tilles, in *Product Operations*, make the same case from the operations side: one of their three pillars of product ops is process and governance, alongside business and data insights and customer and market insights. Without dependable rituals for looking at data, customers and plans, decisions get made on anecdote.

Cadence	Ritual	Artifacts reviewed	Decision it produces
Weekly	Product team sync; metrics review	Dashboard, decision log, risk register, discovery log	What we adjust this week; which risks escalate
Biweekly	Sprint review and planning (with engineering)	Backlog, release plan, active PRDs	What ships next; what gets cut
Monthly	Product review with leadership; roadmap refresh	Roadmap, OKR progress, competitive changes, customer insight summary	Roadmap changes; asks for help
Quarterly	Business review (QBR); OKR reset; portfolio review	Business model, OKRs, competitive landscape, stakeholder map, lifecycle stage	Next quarter's bets; staffing; pricing changes
Annually	Strategy refresh and planning	Vision, strategy one-pager, personas, positioning, business case, lifecycle assessment	Where to play, how to win, what to stop
Per event	Launch readiness; go/no-go; post-launch review; sunset gate	Launch plan, PRFAQ, readiness checklist, sunset plan	Ship or hold; continue, pivot or retire

Two rules make the cadence work. First, **every ritual produces a decision or it gets cancelled.** A weekly metrics review that never changes anything is theater. Second,

decisions get written down, in a decision log with the date, the decision, the options considered, who decided and why. Six months later, when someone asks why you chose usage pricing, the log answers in thirty seconds. Chapter 29 covers decision logs and DACI (the Driver, Approver, Contributors, Informed model popularized by Atlassian) in detail.

FOR PRODUCT LEADERS **For product leaders: nesting the 360**

In a portfolio, Product 360s nest. A group PM or director keeps a portfolio-level 360 whose strategy and business dimensions summarize the products beneath it, and whose operate dimension runs the portfolio review. Each product keeps its own 360. Feature-area PMs often keep a light 360 (customer, build, measure) that inherits strategy from the product level. The rule: a lower-level 360 should never contradict a higher one without a recorded decision.

How to build your Product 360

The method has six steps. The Capstone expands them into a full 30/60/90-day plan with prompts.

1. **Scope the surface area.** Write down, in one paragraph, exactly what you own: which product or products, which customers, which teams, which revenue. Scope disputes are a common hidden cause of a broken 360. If two PMs both think they own pricing, neither of their 360s is true.
2. **Inventory what exists.** Gather every document, deck, dashboard and ticket that describes the product. Don’t judge it yet; just collect it. An AI harness with connectors into your doc tools makes this much faster.
3. **Map it to the nine dimensions.** Sort the inventory by dimension and kind (durable, living, event). Mark each item as current, stale or contradicted. Gaps become obvious immediately.

4. **Build the spine.** Create or repair the twelve spine artifacts, in order, starting with the Product 360 index. Reuse what's good; rewrite what's stale. Each artifact gets an owner and a review date.
5. **Set the cadence.** Put the rituals on the calendar, with the artifacts each one reviews. Start lighter than you think you need. A weekly metrics review and a monthly roadmap review beat an ambitious calendar nobody attends.
6. **Publish and keep it alive.** Share the index with your team, your manager and your key stakeholders. Tell them it's the source of truth. Then make it true: when a decision happens, it goes in the log; when a strategy changes, the one-pager changes first.

The AI lens

Your 360 is your AI context

In 2026, AI turns a well-kept Product 360 into reusable context. When your artifacts are current and readable, you can hand them to a harness and ask questions that used to take a week of meetings:

- "Does the Q3 roadmap still trace back to the strategy one-pager? List anything on the roadmap that doesn't."
- "Summarize every decision in the log that touches pricing, and flag any that contradict the current packaging doc."
- "Draft a first version of the QBR narrative from the dashboard export, the OKR tracker and this month's customer insight summary."

None of those prompts works on a 360 that lives in people's heads. All of them work on a 360 that lives in a connected set of pages. That's a strong practical argument for building the 360 even if you never have time to read every artifact yourself: the harness will.

THE AI LENS **What changes when the product is AI-powered**

Add four artifacts to the spine for any AI-powered product: an **eval plan** (how quality is measured, and the current scores), an **AI cost model** (cost per task or per user, and its trend), a **model and vendor register** (which models are in use, their versions and deprecation dates), and an **AI risk assessment** (failure modes, guardrails, and regulatory obligations). Chapter 31 has templates for all four. They belong in the Measure, Business, Lifecycle and Operate dimensions respectively, which shows how far AI reaches across the dimensions.

What not to hand to AI

The harness drafts, checks and summarizes. It doesn't decide, and it doesn't own. Three guardrails:

- **Data boundaries.** Customer data, unreleased financials and confidential strategy go only into tools your company has approved for that data. Check before you connect.
- **Decisions stay human.** An AI can lay out pricing options with their tradeoffs. You and your approvers choose. The decision log records who decided, and it shouldn't say "the model."
- **Verify the numbers.** AI-generated summaries of metrics are fast and frequently wrong in small ways: a misread column, a wrong time window, a silently dropped segment. Every number that goes into a durable artifact or an executive review gets checked against the source. Chapter 19 covers how.

The Beacon case: a new Principal PM's first inventory

BEACON CASE **Day one on Beacon**

You've just joined Northlight Systems as Principal PM for Beacon, the AI agent platform for IT operations. Your manager, the VP of Product for the Operations portfolio, gives you two weeks to "get your arms around it" before the quarterly business review.

You start with step 1 and write the scope paragraph: "*Beacon: the standalone AI incident-management product, including Beacon Classic (legacy correlation) and Beacon Agents (diagnosis and supervised remediation). Customers: about 420 enterprise accounts plus the MSP partner channel. Teams: 3 PMs, 1 designer, 5 engineering teams, 2 applied AI engineers, a PMM and shared product ops. Revenue: about $48M ARR.*" You send it to your manager and to the group PM for Northlight Observe, because you suspect the boundary between Beacon and Observe is fuzzy. It is: Observe's PM believes alert correlation belongs to Observe. That disagreement goes into the decision log as the first open item.

Step 2, the inventory, turns up 140 documents. Sorted against the nine dimensions, the picture looks like this:

Dimension	What exists	State
Customer	Three personas from 2024, all users, no buyer or security persona	Stale; missing Marcus (economic buyer) and Sam (security reviewer)
Strategy	A 2025 strategy deck; no one-pager; vision statement in an all-hands slide	Stale; doesn't mention agents
Business	Pricing page; a finance model owned by FP&A	Contradicted: gross margin dropped from 81% to 72% and the strategy deck still assumes 80%
Build	Jira roadmap view; 23 PRDs, 9 for features that shipped differently	Living but noisy
Measure	A dashboard with 40 charts; no North Star	No agreed definition of success
Go-to-market	Positioning from the Classic era; no launch tiering	Stale; Beacon Agents launch at Northlight Live is in four months
Operate	Weekly team sync; no decision log; risk register abandoned in 2025	Missing
Lifecycle	Nothing on Beacon Classic's end of life	Missing, and customers are asking
Leadership	No stakeholder map; Observe boundary disputed	Missing

The gaps rank themselves. Gross margin and the missing North Star are business-critical and visible to executives within weeks. The Beacon Agents launch has a fixed date. Beacon Classic's end of life is creating customer escalations. So your first spine artifacts after the index are the business model summary, the North Star and metric tree, the launch plan, and a lifecycle assessment for Classic. Personas and strategy follow, but they can wait a month; the four urgent artifacts can't.

By the QBR you have a Product 360 index with nine dimensions, twelve linked artifacts (four new, five repaired, three still marked “stale, owner assigned”), a decision log with six entries and a cadence on the calendar. It isn’t complete. It’s true, and everyone can see which parts aren’t done yet. That’s the state you’re aiming for.

Product 360 artifacts

TEMPLATE **Product 360 index**

```
# [Product name] · Product 360

**Owner:** [Name, title]   **Last reviewed:** [YYYY-MM-DD]   **Lifecycle stage:**
[Introduce / Grow / Mature / Decline / Retire]

## Scope
[One paragraph: which products, customers, teams and revenue this 360 covers. Name
what is explicitly out of scope and who owns it.]

## The 360 at a glance
| Dimension | Key question | Artifact (link) | Owner | Kind | Last reviewed | State
|
|---|---|---|---|---|---|---|
| Customer | Who is this for, and what job? | [Personas and problem hypothesis] | |
Durable | | Current / Stale / Missing |
| Strategy | Where we play, how we win | [Vision and strategy one-pager] | | Durable
| | |
| Strategy | Who we compete with | [Competitive landscape] | | Living (Q) | | |
| Business | How it makes money | [Business model and unit economics] | | Living (Q)
| | |
| Build | What's next | [Outcome roadmap] | | Living (M) | | |
| Measure | How we know it works | [North Star, metric tree, dashboard] | | Living
(W) | | |
| Measure | What we committed to | [OKRs] | | Living (Q) | | |
| Go-to-market | How it reaches customers | [Positioning and messaging] | | Durable
| | |
| Operate | How we run it | [Operating cadence, decision log, risk register] | |
Living (W) | | |
| Lifecycle | Where it is in its life | [Lifecycle stage assessment] | | Durable | |
|
| Leadership | Who must be aligned | [Stakeholder map and DACI] | | Living (Q) | | |
| AI (if applicable) | Quality, cost, models, risk | [Eval plan, AI cost model,
model register, AI risk assessment] | | Living (M) | | |
```

```
## Current priorities
1. [The one to three things that matter most this quarter, each linked to its artifact]

## Open decisions
[Link to the decision log, filtered to open items]

## Active events
[Links to in-flight PRDs, launch plans, business cases, sunset plans]

## Key contacts
[Engineering lead, design lead, PMM, product ops, sales lead, support lead, finance partner]
```

TEMPLATE **Artifact inventory and gap assessment**

```
# Artifact inventory · [Product] · [Date]

## Scope statement
[Paste the one-paragraph scope.]

## Inventory
| # | Document | Link | Dimension | Kind (durable/living/event) | Last updated |
Owner | State (current/stale/contradicted) | Notes |
|---|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |

## Gaps by dimension
| Dimension | Spine artifact | Exists? | State | Business risk if missing (H/M/L) |
Deadline driver | Priority |
|---|---|---|---|---|---|---|
| Customer | Problem hypothesis and personas | | | | | |
| Strategy | Vision and strategy one-pager | | | | | |
| Strategy | Competitive landscape | | | | | |
| Business | Business model and unit economics | | | | | |
| Build | Outcome roadmap | | | | | |
| Measure | North Star and metric tree | | | | | |
| Measure | OKRs | | | | | |
| Go-to-market | Positioning and messaging | | | | | |
| Operate | Cadence and decision log | | | | | |
| Lifecycle | Lifecycle stage assessment | | | | | |
| Leadership | Stakeholder map and DACI | | | | | |

## Contradictions found
[Each pair of artifacts that disagree, what they disagree on, and who needs to
resolve it.]
```

```
## Build order
1. [Artifact] because [deadline or risk]
```

▤ TEMPLATE

Product operating cadence

```
# Operating cadence · [Product] · [Quarter]

| Cadence | Ritual | Day and time | Attendees | Artifacts reviewed | Output | Owner |
|---|---|---|---|---|---|---|
| Weekly | Metrics review | | PM, eng lead, design, analyst | Dashboard, risk register | Adjustments, escalations | |
| Weekly | Team sync | | Product team | Decision log, discovery log | Decisions logged | |
| Biweekly | Sprint review and planning | | Product team | Backlog, release plan | Sprint commitment | |
| Monthly | Product review | | Leadership, PM, eng, design, PMM | Roadmap, OKRs, competitive changes, insight summary | Roadmap changes, asks | |
| Quarterly | Business review | | Leadership, finance, sales, CS | Business model, OKRs, lifecycle stage | Bets, staffing, pricing | |
| Annually | Strategy refresh | | Leadership | All durable artifacts | Strategy for the year | |
| Per event | Launch go/no-go | | Launch team | Launch plan, readiness checklist | Ship or hold | |

## Rules
- Every ritual produces a decision or is cancelled after two meetings without one.
- Decisions are written in the decision log within 24 hours.
- Each artifact shows its owner and last-reviewed date. Anything past its review date is marked stale on the 360 index.
```

Prompts

✦ PROMPT **Inventory and classify existing product documents**

Goal: Turn a pile of existing documents into a first Product 360 inventory.
Paste in (or connect the harness to): the documents, decks, pages and dashboards you've gathered for [product]. Use only tools approved for this data.

You are helping a Principal PM build a Product 360 for [product]. The Product 360 has nine dimensions: Customer, Strategy, Business, Build, Measure, Go-to-market, Operate, Lifecycle, Leadership. Artifacts are durable (change yearly), living (change on a weekly to quarterly cadence) or event (created for one decision or moment).

For every document provided:
1. Give its title, link or file name, and last-updated date if visible.
2. Assign one primary dimension and the artifact kind.
3. Summarize in one sentence what question it answers.
4. Mark it current, stale (older than its natural cadence) or contradicted (conflicts with another document), and say which document it conflicts with.

Then produce:
- A table of the twelve spine artifacts (Product 360 index, problem hypothesis and personas, vision and strategy one-pager, competitive landscape, business model and unit economics, outcome roadmap, North Star and metric tree, OKRs, positioning and messaging, operating cadence and decision log, lifecycle stage assessment, stakeholder map and DACI) showing which exist, their state and the best existing source for each.
- A list of every contradiction you found, quoting the conflicting statements.

Do not invent documents or fill gaps with guesses. If you can't tell a document's date or state, say so.

Check the output: open three documents the model marked "current" and two it marked "contradicted" and confirm it read them correctly. Spot-check that no spine artifact was marked "exists" on the strength of a passing mention.

✦ PROMPT **Draft the Product 360 index from the inventory**

```
Goal: Produce the first version of the Product 360 index page.
Paste in: the scope paragraph, the completed artifact inventory and the list of open decisions.

Using the inventory below, draft a Product 360 index for [product] using this structure: scope; a table of the nine dimensions with key question, linked artifact, owner, kind, last reviewed and state; current priorities (no more than three); open decisions; active events; key contacts.

Rules:
- Use only artifacts and facts that appear in the inventory. Where an artifact is missing, write "Missing" and leave the owner blank for me to fill.
- Rank current priorities by business risk and deadline, and give a one-line reason for each.
- Keep the page under 600 words. It's a map, not a report.

[inventory]

Check the output: every link should resolve to a real document, every "current" label should match the inventory, and the priorities should be ones you'd defend to your manager.
```

✦ PROMPT **Design an operating cadence that fits the team**

Goal: Get a realistic first cadence instead of an aspirational one.
Paste in: team size and roles, existing recurring meetings, the planning calendar (QBR dates, annual planning), and upcoming events such as launches.

I own [product]. Here's my team and the meetings that already exist: [details]. Propose an operating cadence using the fewest new meetings possible. Reuse existing meetings where they can absorb a ritual. For each ritual give: frequency, attendees, the artifacts reviewed, the decision it produces and a 1-line agenda. Flag any ritual that has no clear decision output, and recommend cutting it. Finish with the total hours per week the cadence costs the PM and the engineering lead.

Check the output: count the hours yourself. If the PM's ritual load exceeds about 20% of the week, ask for a lighter version.

✦ PROMPT **Audit the 360 for contradictions**

```
Goal: Find places where your artifacts disagree before an executive does.
Paste in: your current strategy one-pager, roadmap, OKRs, business model summary and positioning doc (or point the harness to them).

Compare these artifacts for [product] and list every contradiction or unsupported link. Specifically:
1. Roadmap items that don't trace to a strategic choice in the one-pager.
2. OKRs that measure something the strategy doesn't prioritize, or priorities with no OKR.
3. Positioning claims the roadmap doesn't support.
4. Business-model assumptions (margin, price, growth) that conflict with the latest numbers.
For each, quote both sources, explain the conflict in one sentence and suggest which artifact is more likely to be wrong and why.

Check the output: for each contradiction, decide yourself which artifact to change, and log the decision.
```

Anti-patterns

ANTI-PATTERN **The document museum**

A wiki full of beautifully formatted pages, none touched since the launch that justified them. The tell: no artifact shows an owner or a last-reviewed date. Fix it by adding freshness to the index and archiving anything nobody claims.

ANTI-PATTERN **The 360 in your head**

Everything is current, and all of it lives in the PM's memory. The product works until the PM goes on vacation, changes teams or gets asked the same question by two executives on the same day. If it isn't written, it isn't part of the 360.

ANTI-PATTERN **Everything is living**

Strategy, personas and positioning get revised every sprint because a customer said something interesting. Durable artifacts are supposed to be stable. Capture the signal in the discovery log, and change durable artifacts at the strategy refresh, or with an explicit decision.

ANTI-PATTERN **The AI-generated 360 nobody checked**

A harness can produce a complete-looking 360 in an afternoon. If you didn't verify the facts and make the choices, it's a plausible fiction with your name on it. Use AI to draft and to check; own every claim that ships.

ANTI-PATTERN **Scope fog**

Two PMs each believe they own the same capability. Both 360s look complete, and both are wrong. Write the scope paragraph first and get the boundary agreed before you build anything else.

Checklist

- [ ] I've written a one-paragraph scope for my product, and my manager and neighboring PMs agree with it.

- [ ] Every existing document is inventoried and mapped to a dimension and artifact kind.
- [ ] The Product 360 index exists, links every spine artifact and shows owners and last-reviewed dates.
- [ ] All twelve spine artifacts exist, or are explicitly marked missing with an owner and date.
- [ ] For an AI-powered product, the eval plan, AI cost model, model register and AI risk assessment are on the index.
- [ ] Contradictions between artifacts are listed, and each has a decision owner.
- [ ] The operating cadence is on the calendar, and every ritual names the decision it produces.
- [ ] A decision log exists and has entries from the last two weeks.
- [ ] The index is shared with my team, my manager and key stakeholders as the source of truth.
- [ ] The 360 lives somewhere an approved AI harness can read it.

Key takeaways

- A Product 360 is the set of artifacts and rituals that keep a product legible: nine dimensions, each answering a question someone will eventually ask you.
- The dimensions group into Direction (customer, strategy, business), Delivery (build, measure, go-to-market) and Durability (operate, lifecycle, leadership). PM training often skips the third group.
- Artifacts come in three kinds. Durable ones change yearly, living ones change on a cadence and event ones serve a single decision. Treating them all alike produces either a stale strategy or no strategy.
- Start with a spine of twelve artifacts and the Product 360 index. Build everything else when a question demands it.
- The operating cadence keeps the 360 true. Every ritual produces a decision, and every decision gets logged.

- AI isn’t a tenth dimension. It changes all nine, both in what you build and in how you maintain the 360, and a well-kept 360 is the best context you can give a harness.

PART I · PRODUCT MANAGEMENT

01

Developing a Product Hypothesis and Personas

Frame the problem, name the customer, and write a hypothesis you can kill.

35 min read · 8 templates · 6 prompts

Every product starts as a guess about a person, a problem and a solution. The job of this chapter is to make that guess explicit, split it into parts that can each be proven wrong, and replace the parts that fail with evidence before anyone writes a line of code.

Why this matters

Plenty of failed products were built well. They failed because the team built something good for a problem nobody had badly enough to pay for, or for a customer who wasn't the one making the decision. Those are hypothesis failures, and they're cheap to catch early and very expensive to catch late.

The trap is that a product idea feels like one thing. "An AI agent that diagnoses incidents for SRE teams" sounds like a single proposition. It's actually at least three separate bets stacked on top of each other: that SRE teams are the right customer, that slow diagnosis is a problem they'd pay to fix, and that an AI agent is a solution they'd trust. Any one of those can be false while the other two are true. If you test the bundle, a failure tells you nothing about which part broke.

This chapter gives you four tools that work together:

1. **The product hypothesis**, which splits the bet into customer, problem and solution, so each can fail on its own.
2. **Jobs to be done**, which tells you what progress the customer is really trying to make, and what they'd fire to hire you.
3. **Personas**, which turn "the customer" into specific people with specific goals, including the people who buy rather than use.
4. **Journey maps and discovery interviews**, which put the hypothesis in front of reality on a weekly rhythm.

These are the Customer dimension of your Product 360. Everything downstream (vision, strategy, roadmap, positioning, pricing) assumes they're right, so they're the first durable artifacts worth getting right.

The product hypothesis

Three parts that fail separately

A good product hypothesis has three parts, written so each one can be tested independently:

- **Customer hypothesis**: who specifically has the problem, in what role and context.
- **Problem hypothesis**: what hurts, in what situation, and what it costs them.
- **Solution hypothesis**: what you propose, and what observable signal will tell you it works.

The order matters, and teams often get it wrong. If the customer and problem hypotheses fail, the solution hypothesis is moot, so you test them first. Most discovery interviews should test only the first two. Showing someone your solution before you've confirmed their problem invites them to be polite about it, and polite feedback is worthless.

KEY IDEA **Test the problem before the solution**

A solution can only be as right as the problem it solves. Spend your first interviews proving the customer and problem hypotheses. Bring the solution in once you have evidence the problem is real, frequent and costly.

This three-part shape draws on Eric Ries's *The Lean Startup*, which treats every new venture as resting on **leap-of-faith assumptions**. Ries names two that matter most: the **value hypothesis** (does the product deliver value once people use it?) and the **growth hypothesis** (how will new customers discover and adopt it?). In consumer products the growth hypothesis is often about virality. In B2B it's almost always about the buying process: "security teams will approve a self-serve trial," or "the champion can get budget without a new line item." Write that assumption down too, because in enterprise products it often kills ideas that customers value.

The Lean UX hypothesis statement

When you move from problem to solution, Jeff Gothelf and Josh Seiden's *Lean UX* offers a compact format that links a feature to two outcomes at once. It's commonly written as:

> *We believe that* **[business outcome]** *will be achieved if* **[user]** *attains* **[benefit]** *with* **[feature]**.

The value of this format is that it forces you to name both the user's outcome and the business's outcome. In enterprise products, name the customer's outcome and your own separately. "Reduced audit-prep time for the customer" and "expansion revenue for us" are different outcomes, and a feature can deliver one without the other.

Pair each hypothesis with the smallest test that could falsify it and the signal you'll watch. A hypothesis without a falsifying signal is a wish.

Assumption mapping

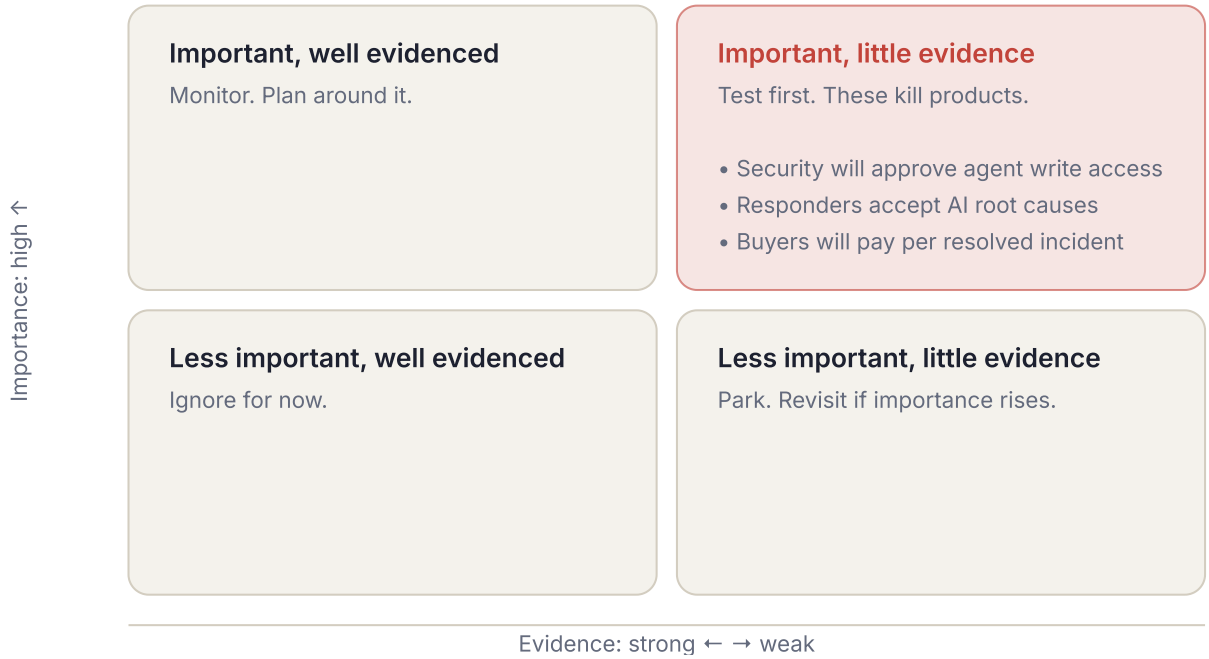
Once you've written the hypothesis, list every assumption underneath it. There are always more than you think. David Bland's assumption-mapping technique, described in *Testing Business Ideas* (with Alex Osterwalder), sorts them two ways.

First, by type of risk:

- **Desirability**: do customers want this?
- **Feasibility**: can we deliver it?
- **Viability**: should we, financially?

Then, on a 2x2: how **important** the assumption is to success, against how much **evidence** you already have. The top-right quadrant (important, little evidence) is where you test first. Everything else can wait.

For B2B products, this book adds a fourth lens that the standard three tend to hide: **adoptability**. Will the buying group, the security review and procurement let this through? A product can be desirable to users, feasible to build and viable on paper, and still never get past a security questionnaire. If your product touches production systems, customer data or AI decision-making, adoptability assumptions usually belong in the top-right quadrant.



An assumption map for Beacon Agents. The top-right quadrant gets tested first. Two of the three assumptions there are adoptability and trust assumptions, not feature assumptions.

The opportunity assessment

Before a team commits real discovery time, it's worth a one-page gate. Marty Cagan's opportunity assessment, first published on SVPG in 2006, asks ten questions:

1. Exactly what problem will this solve?
2. For whom do we solve that problem?
3. How big is the opportunity?
4. What alternatives are out there?
5. Why are we best suited to pursue this?
6. Why now?
7. How will we get this product to market?
8. How will we measure success and make money from it?
9. What factors are critical to success?
10. Given the above, what's the recommendation?

Cagan has since argued, in *Inspired*, that most product work needs lighter framing than a full assessment. Treat the ten questions as a gate for new products and major bets, and use the hypothesis and assumption map for everything smaller. The useful discipline is question 4. Teams that can't name the alternatives (including "do nothing" and "build it ourselves") haven't understood the problem yet.

Jobs to be done

A persona tells you who the customer is. Jobs to be done (JTBD) tells you why they'd buy anything at all. It's one of the most useful ideas in customer understanding, and it comes in two distinct schools that are worth keeping separate.

The core idea

Clayton Christensen and his co-authors, in *Competing Against Luck* and the 2016 HBR article "Know Your Customers' 'Jobs to Be Done'," define a job as **the progress a customer is trying to make in a particular circumstance**. Jobs have functional, social and emotional dimensions. Customers "hire" a product to do the job and "fire" whatever they were using before.

The famous example is a fast-food chain's milkshake. Researchers found that many morning buyers were hiring a milkshake to make a long, boring commute bearable and keep them full until mid-morning. The real competition wasn't other milkshakes. It was bagels, bananas and boredom. The lesson: segment by job and circumstance, not by demographics or product category.

In enterprise software, asking "what did the customer fire to hire us?" is often revealing. The fired option is more often a spreadsheet, a services contract, an internal script or a person's evenings than a competitor.

School one: progress and switching (Moesta, Spiek, Klement)

Bob Moesta and Chris Spiek developed the **four forces of progress**, which explain when a customer actually switches:

- **Push** of the current situation: the friction that starts the search.
- **Pull** of the new solution: the better future they imagine.
- **Anxiety** about the new solution: fear it won't work or will cause new problems.
- **Habit** of the present: the comfort of the way things are.

A switch happens only when push and pull together outweigh anxiety and habit. The model is practical because it shows where effort goes missing. Product teams tend to spend most of their energy on pull (features) and little on reducing anxiety, which in B2B is often the force that decides the deal.

Moesta's **switch interview** reconstructs the timeline of a real, recent purchase, from the first thought that something had to change through looking, deciding and using, attaching each force to dated moments. Interview customers who switched to you in the last 90 days and customers who switched away. In enterprise, interview the champion and at least one other member of the buying group, because the anxiety usually sits with security or procurement, not with the user.

Alan Klement, in a 2013 post on the Intercom blog (crediting Intercom's Paul Adams with the idea), proposed **job stories** as an alternative to persona-based user stories. A job story separates the situation and motivation from the solution:

> **When** *[situation]*, **I want to** *[motivation]*, **so I can** *[expected outcome]*.

Klement's argument is that a user story ("As an SRE, I want an AI root cause…") ties the implementation to the motivation, so when a feature fails you can't tell whether the build or the assumption was wrong. Write the job story first to define the problem. Then write user stories beneath it as candidate solutions.

School two: the job as a process (Ulwick)

Tony Ulwick's **Outcome-Driven Innovation** treats the job as a process with measurable steps. Lance Bettencourt and Ulwick's 2008 HBR article "The Customer-

Centered Innovation Map" lays out a universal job map that any job can be broken into: defining what's needed, locating inputs, preparing, confirming readiness, executing, monitoring, modifying and concluding.

At each step, customers have **desired outcomes**, written in a precise format: a direction of improvement, a metric, the object of control and a contextual clarifier. For example: "Minimize the time it takes to identify which recent change caused an outage." Because the statements are precise and solution-free, you can survey customers on them at scale. Ulwick's **opportunity score** then ranks them:

> $Opportunity = Importance + max(Importance - Satisfaction, 0)$

Importance and satisfaction are the share of respondents rating an outcome as very or extremely important, or very or extremely satisfied, usually scaled to 0 to 10. The `max()` keeps over-served outcomes (high satisfaction) from dragging the score below their importance. High scores mean important and underserved: that's where to build.

Which school to use

The two schools have publicly disagreed, and a Principal PM should be explicit about which lens they're using. In practice they fit different questions:

Question	Use	Why
Why do customers switch to us, or away?	Switch interviews and four forces	Reveals the anxiety and habit your features don't address
What problem should this feature solve?	Job stories	Keeps the motivation separate from the implementation
Which unmet needs matter most across a large market?	ODI job map and opportunity scores	Quantifies importance and satisfaction at scale
Where in a workflow are we losing customers?	Job map plus journey map	Shows which step fails and what it costs

ON-RAMP **If JTBD is new to you**

Start with one habit: before any feature discussion, write the job story. “When [situation], I want to [motivation], so I can [outcome].” If the team can’t agree on the job story, they’re not ready to debate the feature.

Personas

Where personas came from, and why they go wrong

Alan Cooper introduced personas to software design in *The Inmates Are Running the Asylum* (1999). His point was simple: design for a specific person, not an elastic “user” who can be stretched to justify any feature. John Pruitt and Jonathan Grudin later described persona practice at Microsoft and why it works: teams engage with a specific, named person far more than with an abstract user segment. Pruitt and Tamara Adlin’s *The Persona Lifecycle* added the idea that personas are born, used and eventually retired, like any other artifact.

Personas go wrong in predictable ways. They get made once, printed on a poster and never updated. They describe demographics (“Priya, 34, likes hiking”) instead of goals and behaviors. And they get invented in a conference room, then treated as research.

Three kinds of persona

The Nielsen Norman Group distinguishes three kinds, and knowing which one you have is most of the battle:

Type	Basis	Effort	Use it for
Proto-persona	The team's assumptions, no new research	2 to 4 hour workshop	Exposing disagreement and creating hypotheses to test
Qualitative persona	5 to 30 interviews, coded into themes	Weeks	Most product decisions. NN/g calls this the best fit for most teams.
Statistical persona	Qualitative research plus a survey of 100+ (ideally 500+) respondents, clustered	Months	Pricing, packaging and sizing decisions where segment size matters

Jeff Gothelf's **proto-persona** technique, described in UX Magazine in 2012, is especially useful in enterprise companies. Executives and team members each sketch the customer they're picturing on a four-quadrant sheet (a sketch and name; behaviors and beliefs; demographics; needs and goals), present them, and consolidate to three to five. The exercise rarely produces accurate personas. What it reliably produces is the discovery that sales, product and the executive team are picturing three different customers. That's worth knowing on day one.

The practical path: start with proto-personas, label them as hypotheses, and graduate to qualitative personas within a quarter. Save statistical personas for decisions that need segment sizes.

Personas in B2B: the buying group

Consumer products often have one persona who discovers, buys and uses the product. Enterprise products almost never do. The user, the person who signs, the person who can veto and the person who championed the deal internally are different people with different jobs and different anxieties.

Marketing researchers have studied this for more than fifty years. Frederick Webster and Yoram Wind's 1972 model of the **buying center** identified the roles of users, influencers, buyers, deciders and gatekeepers. Robert Miller and Stephen Heiman's *Strategic Selling* popularized the economic buyer, user buyer, technical buyer and coach, and the MEDDIC sales method renamed the coach as the **champion**. Gartner's

research on B2B buying describes six buying jobs that the buying group has to complete, in no fixed order: problem identification, solution exploration, requirements building, supplier selection, validation and consensus creation.

For product work, this book recommends a standard B2B persona set. Every enterprise product should have at least these, each with a job story and its main anxiety:

Persona	Role in the deal	Typical job	Typical anxiety
End user	Uses the product daily	Get their own work done faster or better	"This will make my job harder before it makes it easier."
Champion	Advocates internally and builds the case	Solve a team problem and look good doing it	"If this fails, it's my reputation."
Economic buyer	Owns the budget and signs	Hit a business target within budget	"I can't prove the ROI to my CFO."
Technical evaluator	Security, IT or architecture review	Keep the environment safe and supportable	"This creates a risk I'll be blamed for."
Procurement or legal (gatekeeper)	Contract and vendor approval	Reduce commercial and legal risk	"These terms expose us."

Map each of Gartner's buying jobs to the persona who owns it. Consensus creation is where enterprise deals stall, so the champion needs artifacts that help them sell internally: an ROI model, a security pack, a reference customer. Those artifacts are product decisions as much as sales ones.

FOR PRODUCT LEADERS

Persona sets should be shared across a portfolio, not reinvented by every PM. If three products sell to the same economic buyer, they should agree on who that buyer is and what they need. Make the portfolio's persona set a durable artifact owned at the group level, and let each product extend it with its own end users.

Journey maps

A persona tells you who. A job tells you why. A **journey map** tells you what happens, step by step, as that person tries to make progress, and where it goes wrong.

NN/g's Sarah Gibbons defines a journey map as a visualization of the process a person goes through to accomplish a goal, and lays out its anatomy in three zones:

- **The lens**: one actor (one persona per map) and the scenario, with their expectations.
- **The experience**: the phases of the journey, and for each phase the person's actions, thoughts and emotions (usually drawn as a line of highs and lows).
- **The insights**: the opportunities the map reveals, each with an internal owner and a metric.

Journey maps sit in a family of related tools. An **experience map** is broader and product-agnostic. A **service blueprint**, a technique G. Lynn Shostack introduced in HBR in 1984, extends the journey into the company: the frontstage actions customers see, the backstage actions they don't and the support processes underneath, separated by lines of interaction, visibility and internal interaction. A **user story map** is a planning tool for delivery, covered in Chapter 06.

For B2B products, draw the journey at two levels:

1. **The buying journey**: the buying group working through the six buying jobs, with one swimlane per persona. This is where you discover that the security review

takes eleven weeks.

2. **The usage journey**: one end user, one scenario, the classic NN/g map. This is where you discover that time-to-value is lost in onboarding.

Connect the two at the **first value** moment: the point where the customer first gets the outcome they bought. In enterprise products, a service blueprint of onboarding often shows that the time to first value is lost in handoffs between sales, provisioning, support and customer success, not in the product's interface.

Discovery interviews

Hypotheses, jobs, personas and journeys are all guesses until you talk to customers. Three bodies of work define how to do that well.

The Mom Test

Rob Fitzpatrick's *The Mom Test* is named for the idea that you should be able to ask your mother about your business idea and get useful answers, even though she loves you and wants to be supportive. It has three rules:

1. Talk about their life instead of your idea.
2. Ask about specifics in the past instead of generics or opinions about the future.
3. Talk less and listen more.

Fitzpatrick identifies three kinds of bad data: **compliments** ("that sounds great"), **fluff** (generic claims, hypotheticals and promises about the future) and **ideas** (feature requests, which you should dig behind to find the problem). The real signal is **commitment**: time, reputation or money. In enterprise products, a pilot, a security-review slot or an introduction to an executive counts as commitment. "We'd definitely use that" doesn't.

The single most useful substitution in discovery: replace "Would you use X?" with "Tell me about the last time you had to do Y. What did you use? What did it cost you?"

Continuous discovery

Teresa Torres, in *Continuous Discovery Habits*, argues that discovery shouldn't be a phase. It should be a habit: weekly touchpoints with customers, by the team building the product, in pursuit of a desired outcome. She recommends the **product trio** (PM, designer and tech lead) interview at least one customer a week.

Torres's **opportunity solution tree** organizes what you learn:

- The **desired outcome** is the root: the metric the team is trying to move.
- **Opportunities** branch from it: unmet customer needs, pain points and desires, discovered in interviews.
- **Solutions** branch from opportunities.
- **Assumption tests** sit under each solution.

The tree keeps teams from jumping from outcome straight to solution. Revisit the opportunity space every three or four interviews.

Torres's **story-based interviews** collect specific past stories ("Tell me about a time when…"), and her **interview snapshot** captures each one on a single page: the participant, quick facts for context, a memorable quote, the opportunities heard, insights not yet actionable, and a small experience map of the story's key moments.

For a Principal PM, the habit is the point. Automate recruiting (in-product intercepts, customer success referrals, a standing research panel) so the trio talks to a customer every week without anyone having to kick off a project.

Interviewing craft

Steve Portigal's *Interviewing Users* covers the craft that the other two assume: building rapport, embracing how the participant sees the world, using silence and follow-up questions, and moving from interviewing to sense-making. Use Portigal for how to run the conversation, Fitzpatrick for what to ask, and Torres for cadence and synthesis.

How to apply it

Here's a six-week method for building the Customer dimension of a Product 360 from scratch, or for testing a major new bet within an existing product.

Week 1: Write it down.

1. Draft the three-part product hypothesis: customer, problem, solution. Include the growth (buying-process) assumption.
2. Run a 90-minute proto-persona workshop with the product trio, a sales leader and a customer success leader. Consolidate to three to five personas, including the B2B roles. Label every persona "hypothesis."
3. Write a job story for each persona.
4. List every assumption and map them: desirability, feasibility, viability, adoptability; importance against evidence.

Week 2: Recruit and prepare.

5. Write an interview guide that tests the customer and problem hypotheses only. Use the Mom Test rules: past behavior, specifics, no pitching.
6. Recruit 12 to 15 interviews across personas: at least five end users, three champions, two economic buyers and two technical evaluators. Include recent switchers to you and away from you.

Weeks 3 to 5: Interview continuously.

7. Run two to five interviews a week as a trio. Write an interview snapshot within 24 hours of each.
8. Every three or four interviews, update the opportunity solution tree and the assumption map. Move assumptions as evidence arrives.
9. For switchers, run the four-forces analysis. Pay special attention to anxiety.

Week 6: Decide.

10. Revise the personas from proto to qualitative, based on what you heard. Retire any that didn't show up in reality.
11. Draw the buying and usage journey maps, and mark the opportunities.

12. Rewrite the hypothesis. For each of the three parts, state whether it's confirmed, revised or killed, with the evidence.
13. Make a decision with your manager: proceed to vision and strategy (Chapter 02), pivot the hypothesis and run another cycle, or stop.

Then keep the weekly interview habit going. The Customer dimension is durable, but the discovery log behind it is living.

The AI lens

Building AI-powered products

When the product is AI-powered, the customer hypothesis needs a few things that classic hypotheses leave out.

- **The trust threshold.** For any AI output, there's a level of accuracy below which users won't rely on it and a level of consequence above which they won't let it act. Make both part of the problem hypothesis. "SRE leads will accept an AI-proposed root cause if it's right at least [x]% of the time and shows its evidence" is testable. "Users want AI diagnosis" isn't.
- **The delegation job.** AI products, especially agents, change the job from "help me do this" to "do this for me." Those are different jobs with different anxieties. Write both job stories and test which one customers are hiring for. Often it's the first for now and the second later.
- **The technical evaluator's new questions.** For AI products, the technical evaluator persona has a growing list of concerns: where data goes, whether it trains models, what the agent can access, how decisions are logged, and what regulation applies. Chapter 33 covers the regulatory side. In discovery, the point is to interview this persona early, because their anxiety decides adoptability.
- **The persona's AI posture.** Add one field to every persona in an AI product: how they feel about AI doing their work today. Enthusiasts, pragmatists and skeptics behave very differently, and the mix shifts from one segment to the next.

Doing the work with AI

AI changes discovery work dramatically, mostly in synthesis. User Interviews' *State of User Research* 2025 found that 80% of researchers were using AI, up sharply from the prior year, while 91% worried about accuracy and hallucinations. Both findings hold at once, and the practical rules follow from them.

Where AI helps. NN/g's Kate Moran and Maria Rosala, in "Accelerating Research with AI," find the biggest gains in planning and analysis: drafting interview guides, transcribing, first-pass coding and clustering of themes, drafting snapshots and summarizing across many interviews. A harness with access to your interview transcripts, support tickets, CRM notes and sales call recordings can draft a first opportunity solution tree far faster than a team working by hand.

Where AI doesn't help. AI can't observe behavior, so it shouldn't moderate or analyze usability tests. It can't decide what a story means for your strategy. And it tends to smooth over exactly the things that matter most in discovery: the rare, surprising remark and the subgroup that behaves differently.

Keep the evidence attached. Every AI-generated insight should link back to the source quote or clip. If you can't trace a theme to at least two real customers, it's the model's opinion, not a finding.

The synthetic users debate

The most contested AI practice in discovery is the **synthetic user**: an AI-generated respondent meant to stand in for a real customer. The evidence is now strong enough to give clear guidance.

NN/g's 2024 analysis by Rosala and Moran found synthetic users to be sycophantic and idealized. They "care about everything," so they can't prioritize; they can't report real past behavior because they don't have any; and they're weakest for niche groups. In October 2026, NN/g published an empirical comparison of 120 synthetic respondents against 604 human respondents to the same survey. The headline conclusions roughly matched, but humans produced more than fifty distinct labels for their own field where the synthetic respondents produced about a dozen: a collapse in variety that hides exactly the edge cases discovery exists to find.

The strongest evidence for AI respondents comes with an important caveat. A 2024 Stanford-led study (Park and colleagues) built AI agents from two-hour interviews with 1,052 real people, and those agents replicated each person's survey answers about 85% as well as the people replicated their own answers two weeks later. But each agent was grounded in an interview with one specific, real person. That's a digital twin of a participant you already researched, not a persona invented from a prompt.

The rules for an enterprise PM:

- **Use synthetic users to prepare.** Pilot an interview guide, explore an unfamiliar domain, or generate a first set of proto-personas to test.
- **Never present synthetic output as research.** Label it hypothesis, and validate it with real customers.
- **Be most skeptical where B2B insight matters most.** Niche roles, buying-group politics and long-tail edge cases are exactly where synthetic users are weakest.

THE AI LENS **The one-line rule**

AI can draft, cluster and summarize what customers said. It can't tell you what customers would say, and it can't decide what it means. Supplement human research with AI; don't substitute for it.

The Beacon case: testing the Beacon Agents hypothesis

BEACON CASE **Where Beacon starts**

Beacon's leadership wants to move from "correlate and summarize" to Beacon Agents: AI agents that diagnose incidents and run approved remediation steps. The strategy deck assumes SRE teams want it. Your Product 360 inventory (Chapter 00) showed that the existing personas are all end users from 2024, with no economic buyer or security reviewer. Before the Agents roadmap gets committed, you test the hypothesis.

The hypothesis. You write it in three parts:

- *Customer*: We believe SRE and platform teams of 20 to 100 engineers at enterprises already using Beacon for alert correlation…
- *Problem*: …lose most of each incident to diagnosis (finding which change, service or dependency caused it), which costs them [time to resolve, on-call burnout and SLA penalties].
- *Solution*: We believe an agent that proposes a root cause with cited evidence, and runs allowlisted remediation steps after human approval, will cut diagnosis time by at least half. We'll know we're right when responders accept the agent's diagnosis in at least 60% of incidents during a 30-day pilot.
- *Growth (buying process)*: Security teams will approve agent write access to production for allowlisted actions.

The assumption map. The trio maps 22 assumptions. Three land in the top-right quadrant: security will approve write access; responders will accept AI root causes; and buyers will pay per resolved incident. None of them is a feature assumption.

The proto-personas. The workshop produces five personas, and an immediate surprise. The sales leader pictures the buyer as a VP of IT Operations; the CTO pictures an SRE lead. Both exist: Marcus (VP of IT Operations) is the economic buyer and Priya (SRE lead) is the champion. Dana (on-call engineer) is the end user, Sam (security and compliance reviewer) is the technical evaluator and Leo (MSP operations manager) is the partner persona. Nobody had previously written down Sam.

The interviews. Over three weeks the trio runs 14 interviews: six on-call engineers, three SRE leads, two VPs, two security reviewers and one MSP. They include two customers who left Beacon for a competitor's AIOps feature.

What they hear:

- **The problem is confirmed, with a twist.** Diagnosis really is most of the incident. But the most-cited cost is on-call burnout and attrition, ahead of MTTR. Marcus cares about MTTR because his CFO does; Priya cares about keeping her team. That changes the positioning conversation in Chapter 24.
- **The solution is half-confirmed.** Engineers want the diagnosis badly ("just tell me which deploy did it"). They are far more cautious about the agent acting. Several describe a specific incident where an automated remediation script made an outage worse.
- **Anxiety lives with Sam.** Both security reviewers say they'd block agent write access without per-action audit logs, scoped credentials and a kill switch. One says they'd also need documentation for EU AI Act obligations, because their company sells in Europe.
- **The two switchers** left because the competitor's feature was bundled into a platform they already paid for. The push was price and procurement simplicity, not capability. That's a strategy signal for Chapter 03.

The decision. You rewrite the hypothesis. The customer hypothesis holds. The problem hypothesis is revised: burnout joins MTTR as a primary cost. The solution hypothesis is split in two. Diagnosis ships first, aiming at a 60% acceptance rate. Supervised action follows, gated on security features (audit logs, scoped credentials, a kill switch) that move from "later" to "now" on the roadmap. The growth assumption is replaced: security will approve write access *if* those controls exist, which becomes the next assumption to test with a design partner.

Sam is now a formal persona in the Product 360, with a job story: "*When a new tool asks for production access, I want to see exactly what it can do and how to revoke it, so I can approve it without becoming the person who gets blamed for the next outage.*"

Product 360 artifacts

TEMPLATE **Product hypothesis**

```
# Product hypothesis · [Product or initiative] · v[n] · [Date]

**Owner:** [PM]   **Status:** Draft / Testing / Confirmed / Revised / Killed

## Customer hypothesis
We believe [specific segment, role and context]...

## Problem hypothesis
...experience [problem] when [situation], which costs them [measurable consequence].

## Solution hypothesis
We believe [solution concept] will solve this.
We will know we are right when [observable signal] reaches [threshold] within
[timeframe].

## Growth or buying-process hypothesis
We believe customers will discover and adopt this by [channel or buying process],
and that [buying-group gate, such as security or procurement] will approve it
because [reason].

## Business outcome
If the hypothesis holds, the business outcome is [metric and target]. The customer
outcome is [metric and target].

## Riskiest assumptions (from the assumption map)
1. [Assumption] · Type: desirability / feasibility / viability / adoptability ·
Test: [smallest test] · Signal: [what would falsify it]
2.
3.

## Evidence log
| Date | Source (interview, data, test) | What we learned | Effect on hypothesis |
|---|---|---|---|
```

```
| | | | |

## Decision
[Proceed / pivot / stop], decided by [name] on [date]. Reason: [one paragraph].
```

▤ TEMPLATE

Assumption map

```
# Assumption map · [Product or initiative] · [Date]

| # | Assumption | Type (D/F/V/A) | Importance (H/M/L) | Evidence today
(strong/some/none) | Quadrant | Test | Owner | Status |
|---|---|---|---|---|---|---|---|---|
| 1 | | | | | | | | |

Types: D = desirability (do they want it?), F = feasibility (can we build it?), V =
viability (should we, financially?), A = adoptability (will the buying group,
security and procurement let it through?).

## Test first (important, little evidence)
1.
2.
3.

## Review rhythm
Update after every three or four interviews or tests. Move assumptions between
quadrants as evidence arrives, and note why.
```

TEMPLATE

Opportunity assessment

```
# Opportunity assessment · [Opportunity] · [Date]

**Owner:** [PM]   **Decision needed by:** [Date]   **Decision maker:** [Name]

1. **Exactly what problem will this solve?** (value proposition)
2. **For whom do we solve that problem?** (target personas and segments)
3. **How big is the opportunity?** (market size and how it was estimated)
4. **What alternatives are out there?** (competitors, substitutes, build-it-
yourself, do nothing)
5. **Why are we best suited to pursue this?** (our differentiator)
6. **Why now?** (market window)
7. **How will we get this product to market?** (go-to-market approach)
8. **How will we measure success and make money?** (metrics and revenue model)
9. **What factors are critical to success?** (solution requirements and riskiest
assumptions)
10. **Recommendation:** go, no-go or investigate further, with the next decision
point.

Adapted from Marty Cagan's ten opportunity-assessment questions.
```

TEMPLATE **B2B persona**

```
# Persona · [Name], [Role]

**Type:** Proto (hypothesis) / Qualitative / Statistical   **Evidence:** [number of interviews, sources]   **Owner:** [PM]   **Last validated:** [Date]   **Retire by:** [Date or condition]

## Role in the deal
End user / Champion / Economic buyer / Technical evaluator / Procurement or legal / Partner

## Context
- Organization type and size:
- Team and reporting line:
- Tools they use today:
- What they're measured on:

## Job story
When [situation], I want to [motivation], so I can [expected outcome].

## Goals
1.
2.

## Pains (in their words)
- "[Quote]" (interview #, date)
- "[Quote]"

## Four forces
- Push (what's wrong today):
- Pull (the better future they imagine):
- Anxiety (what worries them about switching):
- Habit (what keeps them where they are):
```

```
## AI posture (AI products only)
Enthusiast / Pragmatist / Skeptic, with evidence:

## What they need from us to say yes
[Proof points, artifacts, controls, references]

## Buying jobs they own
Problem identification / Solution exploration / Requirements building / Supplier
selection / Validation / Consensus creation
```

TEMPLATE **Job stories and desired outcomes**

```
# Jobs · [Product] · [Date]

## Job stories (one per persona and situation)
| Persona | When (situation) | I want to (motivation) | So I can (outcome) |
Evidence |
|---|---|---|---|---|
| | | | | |

## Job map (for one core job)
| Step | What the customer is doing | Desired outcomes (direction + metric + object
+ context) |
|---|---|---|
| Define | | |
| Locate | | |
| Prepare | | |
| Confirm | | |
| Execute | | |
| Monitor | | |
| Modify | | |
| Conclude | | |

## Opportunity scoring (if surveyed)
| Desired outcome | Importance (0 to 10) | Satisfaction (0 to 10) | Opportunity = I
+ max(I - S, 0) |
|---|---|---|---|
| | | | |
```

TEMPLATE **Customer journey map**

```
# Journey map · [Persona] · [Scenario] · [Date]

**Journey type:** Buying journey / Usage journey   **Evidence:** [sources]

## Lens
- Actor:
- Scenario:
- Expectations:

## Experience
| Phase | [Phase 1] | [Phase 2] | [Phase 3] | [Phase 4] | [Phase 5] |
|---|---|---|---|---|---|
| Actions | | | | | |
| Thoughts | | | | | |
| Emotion (-2 to +2) | | | | | |
| Touchpoints | | | | | |
| Backstage (who at our company is involved) | | | | | |
| Pain points | | | | | |

## First value
The moment the customer first gets the outcome they bought: [describe]. Median time
to reach it today: [time].

## Opportunities
| Opportunity | Phase | Owner | Metric | Priority |
|---|---|---|---|---|
| | | | | |
```

TEMPLATE **Discovery interview guide**

```
# Discovery interview guide · [Topic] · [Date]

**Hypothesis under test:** customer / problem (not solution)   **Persona:**
[persona]   **Length:** 45 minutes

## Before the call
- Learning goals (max three):
  1.
  2.
  3.
- What would change our mind:

## Opening (5 min)
- Thank them, explain we're learning, not selling. Ask permission to record.
- "Tell me about your role and what a typical week looks like."

## The story (25 min)
- "Tell me about the last time [the situation] happened."
- "What happened first? Then what?"
- "Who else was involved?"
- "What did you use to deal with it? Why that?"
- "What did it cost you: time, money, stress, reputation?"
- "What have you tried to fix it? What happened?"
- "If it happened again tomorrow, what would you do?"

## Switching (if they recently changed tools) (10 min)
- "When did you first think something had to change?"
- "What made you start looking? What did you look at?"
- "What almost stopped you?"
- "What would make you switch away?"

## Close (5 min)
- "Is there anything I should have asked?"
```

```
- "Who else should I talk to?"
- Ask for a commitment: a follow-up, an intro, a pilot.

## Rules
Ask about past specifics, not future hypotheticals. Don't pitch. Dig behind feature requests to the problem. Silence is a tool.
```

TEMPLATE **Interview snapshot**

```
# Interview snapshot · [Participant] · [Date]

**Persona:** [persona]   **Interviewers:** [trio]   **Recording:** [link]

## Quick facts
[Context that matters: company, team size, tools, role]

## Memorable quote
"[Quote]"

## The story in five moments
1.
2.
3.
4.
5.

## Opportunities heard (needs, pains, desires)
-
-

## Insights (notable, not yet actionable)
-

## Evidence for or against the hypothesis
| Assumption | For / Against / Neutral | Evidence |
|---|---|---|
| | | |

## Commitment offered
[None / follow-up / intro / pilot / other]
```

Prompts

✦ PROMPT **Pressure-test a product hypothesis**

```
Goal: Find the weakest part of your hypothesis before customers do.
Paste in: your three-part product hypothesis, growth assumption and any evidence you already have.

Act as a skeptical product leader reviewing this hypothesis for [product].

1. Rewrite it so the customer, problem and solution parts can each be proven false on their own. Flag any part that's untestable as written.
2. List at least 15 assumptions underneath it. Classify each as desirability, feasibility, viability or adoptability (will the buying group, security and procurement let it through?).
3. Rate each assumption's importance (H/M/L) and the evidence I've provided (strong/some/none). Show the top-right quadrant: important, little evidence.
4. For the top three, propose the smallest, fastest test and the result that would falsify it.
5. Name the two customer alternatives I seem to be underestimating, including "do nothing."

Don't soften the critique. Don't invent evidence I haven't given you.

[hypothesis and evidence]

Check the output: are the top-three assumptions really the ones that would kill the product? If the model ranked a feature preference above a buying-process or trust assumption, push back.
```

✦ PROMPT **Draft proto-personas from existing company data**

Goal: Get a first set of proto-personas to test, built from what the company already knows.
Paste in (or connect the harness to): CRM opportunity notes, win/loss notes, sales call summaries, support tickets and any existing personas. Use only tools approved for customer data, and remove personal identifiers first.

From the material provided for [product], draft proto-personas for an enterprise buying group: end user, champion, economic buyer, technical evaluator and procurement or legal, plus any role that appears repeatedly.

For each persona give: role in the deal, context, a job story ("When..., I want to..., so I can..."), goals, pains quoted from the material with source references, the four forces (push, pull, anxiety, habit) and what they need to say yes.

Rules:
- Every claim must cite the source document. If the material doesn't support a field, write "No evidence yet."
- Mark every persona "Proto-persona: hypothesis to test."
- Note where different sources seem to describe different people in the same role.

Check the output: read three of the cited sources and confirm the quotes and paraphrases are accurate. Treat the personas as the starting point for interviews, not their conclusion.

✦ PROMPT **Write a Mom Test interview guide**

Goal: Produce an interview guide that tests the problem, not your solution.
Paste in: the customer and problem hypotheses, the persona you'll interview and your three learning goals.

Write a 45-minute discovery interview guide for [persona] to test this customer and problem hypothesis: [hypothesis].

Follow the Mom Test rules: ask about specific past events, not future hypotheticals or opinions; never describe our solution; dig behind any feature request to the underlying problem. Include an opening, a story section built around "Tell me about the last time...", a switching section for people who recently changed tools, and a close that asks for a concrete commitment.

After the guide, list five questions I might be tempted to ask that would produce bad data (compliments, fluff or ideas), and rewrite each one.

Check the output: delete any question that mentions our product or asks "would you." Read the guide aloud; if it sounds like a sales call, rewrite it.

✦ PROMPT **Turn an interview transcript into a snapshot**

Goal: Capture each interview on one page within a day.
Paste in: one interview transcript, the persona and the current list of assumptions under test. Use an approved tool for customer recordings.

Create an interview snapshot from this transcript with these sections: quick facts, one memorable quote (verbatim), the story in five key moments, opportunities heard (needs, pains and desires, phrased from the customer's point of view and free of solutions), insights not yet actionable, evidence for or against each assumption listed, and any commitment offered.

Rules:
- Quote verbatim, with timestamps. Never paraphrase inside quotation marks.
- If the participant talked about the future or gave an opinion rather than a past example, label it "opinion" and weight it lower.
- Don't infer things the participant didn't say.

[assumptions]
[transcript]

Check the output: play the clip at two of the timestamps and confirm the quotes. Make sure the opportunities are problems, not features.

✦ PROMPT **Synthesize interviews into an opportunity solution tree**

Goal: Find the patterns across many interviews without losing the evidence.
Paste in: all interview snapshots so far, the desired outcome (the metric the team is trying to move) and the current assumption map.

Using these interview snapshots for [product], build an opportunity solution tree with this desired outcome at the root: [outcome].

1. Cluster the opportunities heard into parent and child opportunities. Name each from the customer's point of view.
2. For each opportunity, list the participants who raised it (by snapshot ID), with a representative quote, and how many personas it spans.
3. Flag any opportunity raised by only one participant as "single source," and any that appears only in one persona.
4. Identify surprises: opportunities that contradict our hypothesis or appear in an unexpected persona.
5. Update the assumption map: which assumptions gained or lost evidence?

Don't propose solutions yet.

Check the output: every opportunity should trace to at least one real quote. Read the single-source and surprise items yourself; they're where models are most likely to over-generalize, and also where the best insights often hide.

✦ PROMPT **Pilot an interview guide with a synthetic respondent**

```
Goal: Find confusing or leading questions before you spend a real customer's time.
Paste in: your interview guide and a short description of the persona.

Role-play as a [persona description] being interviewed with this guide. Answer each
question in character, briefly. Then step out of character and tell me:
1. Which questions were confusing, leading or likely to produce opinions rather than
past stories.
2. Which questions a real person in this role might refuse or be unable to answer,
and why.
3. What important topic the guide doesn't cover.

Important: your in-character answers are not research and will not be used as
evidence. I only want feedback on the guide.

[guide]

Check the output: use only the guide feedback. Discard the in-character answers
entirely.
```

Anti-patterns

ANTI-PATTERN **The bundled hypothesis**

“Enterprises want an AI agent for incidents” gets tested as one bet. When the pilot disappoints, nobody can tell whether the customer, the problem or the solution was wrong. Split the hypothesis, and test the customer and problem before the solution.

ANTI-PATTERN **Personas as posters**

Beautifully designed personas with stock photos and hobbies, made once and never updated. Personas are artifacts with owners, evidence and retirement dates. If a persona hasn't been validated against a real conversation in six months, mark it stale.

ANTI-PATTERN **The user-only persona set**

Every persona is a user. Nobody wrote down the economic buyer, the security reviewer or procurement, so the product delights users and dies in the buying process. Every enterprise product needs the buying group.

ANTI-PATTERN **Asking about the future**

"Would you use this?" "How much would you pay?" "Is this a problem for you?" Each produces polite, useless answers. Ask about the last time it happened, what they did and what it cost.

ANTI-PATTERN **Synthetic research presented as real**

An AI-generated survey of "500 enterprise buyers" appears in a strategy deck. It's fluent, confident and fictional. Synthetic output is for preparing research, never for replacing it.

ANTI-PATTERN **Discovery as a phase**

The team does six weeks of research before a big build, then stops talking to customers for a year. Make discovery a weekly habit with automated recruiting, so the evidence keeps arriving after the strategy is set.

Checklist

- [ ] The product hypothesis is written in three parts (customer, problem, solution) plus a growth or buying-process assumption, and each part can be falsified on its own.
- [ ] Every assumption is on an assumption map, classified as desirability, feasibility, viability or adoptability, and the top-right quadrant has tests assigned.
- [ ] Major bets have a one-page opportunity assessment with a recommendation and a decision maker.
- [ ] Each persona has a job story, and the core job has a job map with desired outcomes.
- [ ] The persona set includes the full B2B buying group: end user, champion, economic buyer, technical evaluator and procurement or legal.
- [ ] Every persona is labeled proto, qualitative or statistical, with its evidence and last-validated date.
- [ ] Buying and usage journey maps exist and connect at the first-value moment.
- [ ] The interview guide tests the problem, not the solution, and follows the Mom Test rules.
- [ ] At least 12 interviews across personas are complete, each with a snapshot, including recent switchers.
- [ ] The opportunity solution tree traces every opportunity to real quotes.
- [ ] For AI products, the hypothesis states trust thresholds and the personas record AI posture.
- [ ] No synthetic or AI-generated output is presented as customer evidence.
- [ ] The hypothesis has been revised with evidence, and a proceed, pivot or stop decision is in the decision log.
- [ ] A weekly interview habit with automated recruiting is running.

Key takeaways

- A product idea is several bets stacked together. Split it into customer, problem and solution hypotheses, and test the first two before showing anyone the third.
- Map every assumption by type and by importance against evidence. In B2B, add adoptability: the security review and the buying group often kill products that users want.
- Jobs to be done tells you why customers would buy anything. Use switch interviews and the four forces to understand switching, job stories to frame features and Ulwick's outcome scores to prioritize at scale.
- Enterprise personas must include the buying group. The economic buyer, champion, technical evaluator and procurement each have a different job and a different anxiety.
- Discovery is a weekly habit, not a phase. Ask about the past, talk less and treat commitment as the only real signal.
- AI is good at drafting guides and synthesizing transcripts, and unreliable at standing in for customers. Keep every insight linked to a real quote, and label synthetic output as hypothesis.

PART VIII · AI-ERA PRODUCT MANAGEMENT

31

Building AI-Powered Products

AI PRDs, evals, agents, cost, latency, trust and UX.

36 min read · 5 templates · 7 prompts

An AI-powered product doesn't behave the same way twice, costs money every time someone uses it, and has to earn the user's trust one answer at a time. Those three facts change how you specify it, measure it, price it and run it. This chapter shows the product manager what changes in each.

Why this matters

For thirty years, software products were deterministic. You wrote a requirement, engineering built it, QA checked that the same input produced the expected output, and you shipped. Marginal cost was close to zero, so pricing was mostly a question of value and competition. Trust came from reliability: once a feature worked, it kept working.

AI-powered products break all three assumptions.

- **Behavior is a distribution, not a specification.** The same prompt can produce a great answer, a mediocre one and a confidently wrong one. You can’t specify the output; you can only specify how often it must be good, and what “good” means.
- **Cost scales with usage.** Every answer has an inference cost. A feature that delights users can quietly destroy gross margin. Agentic features, which call models many

times per task, make this worse.

- **Trust is earned per answer.** Users can't easily tell a right answer from a plausible wrong one. A single confident mistake in a high-stakes moment can cost more trust than a hundred good answers earn.

The result is that the PM's job on an AI product expands. Quality becomes a product decision you specify with evals. Cost becomes a requirement with a gross-margin floor. Trust becomes a design problem with its own patterns. Autonomy becomes a dial you set per action, not a feature you ship. And regulation, for the first time in most enterprise PMs' careers, becomes a product input rather than a legal review at the end.

This chapter covers each of those. Chapter 32 covers the other half of AI-era product management: using AI to do the PM job itself. Chapter 33 goes deeper on responsible AI and regulation.

Does this need AI at all?

The first question is the one teams skip. Google's People + AI Research (PAIR) team makes it the opening chapter of the *People + AI Guidebook*: decide whether AI adds value before deciding how to build it.

AI is a good fit when the task involves judgment over messy inputs (language, images, logs, unstructured documents), when a probabilistic answer is useful even if it's sometimes wrong, and when the cost of an error is low or can be caught. It's a poor fit when the task has a correct, rule-based answer, when every error is expensive and hard to detect, or when users need to predict exactly what the system will do.

PAIR's framing of **precision versus recall** is a practical tool here. Every AI feature makes two kinds of mistake: false positives (it says yes when the answer is no) and false negatives (it misses something real). They rarely cost the same. For Beacon, a false-positive root cause ("the deploy at 14:02 caused this") can send an engineer to roll back the wrong service, which is expensive. A false negative ("I couldn't determine the cause") just means the engineer diagnoses it manually, which is the status quo.

That asymmetry should shape the product: tune for precision, and make “I don’t know” a first-class answer.

KEY IDEA **Start with the cost of being wrong**

Before choosing a model or a pattern, write down what each kind of error costs the user and the business. That one table will drive your eval targets, your autonomy settings, your UX and, often, your pricing.

Workflows, agents and autonomy

Workflows versus agents

Anthropic’s widely cited essay “Building effective agents,” by Erik Schluntz and Barry Zhang, draws a distinction every AI PM needs. **Workflows** orchestrate models and tools through predefined code paths. **Agents** let the model direct its own process and tool use, in a loop, based on what it observes. Both are “agentic systems,” and they sit on a spectrum of patterns, from simple to complex:

Pattern	What it does	Use it when
Augmented model	One model call with retrieval, tools or memory	The task is a single step with the right context
Prompt chaining	A fixed sequence of model calls, each feeding the next	The task splits into predictable steps
Routing	Classifies the input and sends it to a specialized path	Different inputs need different handling
Parallelization	Splits work into parallel calls, or runs the same call several times and votes	Speed matters, or you want more confidence
Orchestrator-workers	One model breaks the task down and delegates to others	Subtasks can't be predicted in advance
Evaluator-optimizer	One model drafts, another critiques, in a loop	Clear evaluation criteria exist and iteration helps
Autonomous agent	The model uses tools in a loop until it reaches a goal or a stopping condition	The task is open-ended and the steps can't be predicted

The essay's central advice is to start with the simplest pattern that works. Agentic systems trade latency and cost for capability. Many enterprise features work better as workflows, which are cheaper, faster and much easier to evaluate and audit. Use an agent only when the problem really is open-ended, and record the choice in the PRD as a decision with its rationale.

The same essay introduces the **agent-computer interface**: design the tools an agent uses as carefully as you'd design a human interface. Write tool descriptions the way you'd brief a junior developer, and design arguments so mistakes are hard to make. That's product work, and in B2B platforms it often falls to the PM who owns the API.

Autonomy is a dial, set per action

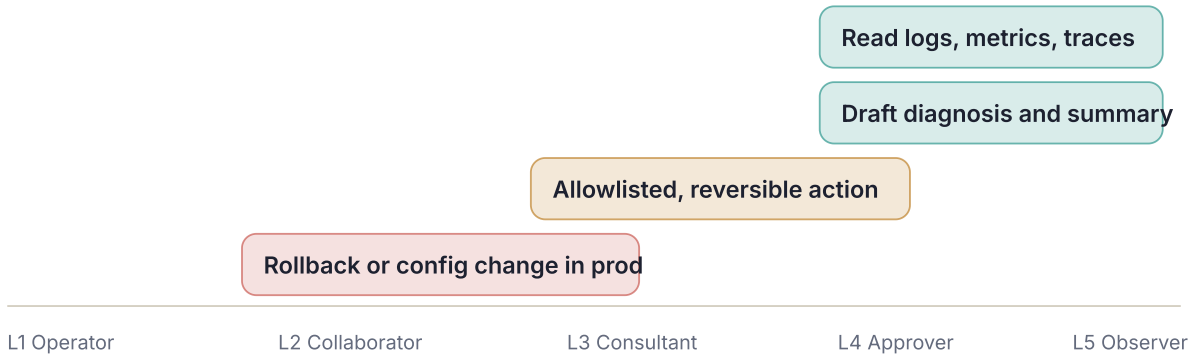
Kevin Feng, David McDonald and Amy Zhang's 2025 paper "Levels of Autonomy for AI Agents" gives product teams a useful vocabulary. It defines five levels by the **user's**

role, not the agent's capability:

Level	User's role	What the agent does
L1	Operator	The user directs and decides; the agent carries out
L2	Collaborator	The user and agent plan and execute together
L3	Consultant	The agent leads and consults the user for expertise or preferences
L4	Approver	The agent acts and involves the user only for risky or pre-specified situations
L5	Observer	The agent acts fully autonomously; the user monitors

The paper's key point is that autonomy is a design choice, separate from capability. A model that *can* act autonomously shouldn't necessarily be *allowed* to.

For product specification, this book recommends setting autonomy **per action class**, not per product. An agent can sensibly be at L4 for reading logs and drafting a diagnosis, and at L2 for anything that writes to a production system. Pair that with a **reversibility map**: which actions can be undone cheaply, which are costly to reverse and which are irreversible. Irreversible actions need approval, dry-run modes or two-step confirmation, however good the model is.



Beacon Agents' autonomy, set per action class. Reading and drafting run at L4; reversible allowlisted actions at L3; anything that changes production at L2, with a human approval on every action.

Connectors, MCP and agent protocols

Agents are only as useful as what they can reach. In 2026 that increasingly runs through open protocols. The **Model Context Protocol (MCP)**, which Anthropic introduced in late 2024, became the common way for AI applications to connect to tools and data. In December 2025 Anthropic donated MCP to the new Agentic AI Foundation under the Linux Foundation, co-founded with OpenAI and Block, and at that point Anthropic reported more than 10,000 active public MCP servers. Google's **Agent2Agent (A2A)** protocol, hosted by the Linux Foundation since mid-2025, addresses a different problem: agents talking to other agents across organizations. The two are complementary, and the landscape is still moving.

For a B2B platform PM, the practical implication is that an **MCP server is now a product surface**, comparable to a public API. Customers' agents (in Claude, ChatGPT, Copilot or their own harnesses) reach your product through it. It needs a scope, an authentication model, rate limits, versioning, documentation, and a pricing position. Treat it as a product, with an owner, rather than as an integration somebody builds on a Friday.

Evals: quality as a product decision

Why evals replace “it works”

In a deterministic product, QA checks whether a feature works. In an AI product, the question is how often it works, on which kinds of input, and how badly it fails when it doesn’t. The tool for answering that is an **eval**: a set of test cases, run automatically, each graded pass or fail, producing a quality score you can track over time and gate releases on.

Evals are where the PM’s definition of quality becomes executable. That’s why Lenny Rachitsky’s 2025 podcast with Hamel Husain and Shreya Shankar billed evals as the hottest new skill for product builders, and why the PM (or a domain expert the PM designates) must own the eval’s definition of “good” rather than leaving it to engineering.

Start with error analysis

Hamel Husain and Shreya Shankar, whose evals course and FAQ are a widely used practitioner reference, put it bluntly: error analysis is “the most important activity in evals.” Their method:

1. **Read real traces.** Pull about 30 real interactions (inputs, model outputs, tool calls). Read every one.
2. **Write free-form notes** on what went wrong in each. Don’t use a rubric yet; rubrics written in advance narrow what you notice.
3. **Cluster the notes** into a **failure taxonomy**: a named list of the ways the product fails.
4. **Keep going until you stop finding new failure types.** That’s your signal that the taxonomy is complete enough for now.

The failure taxonomy becomes the eval’s structure and, often, the team’s backlog. Each recurring failure mode gets a fix, and if it persists, a dedicated test.

Husain and Shankar add several rules that are worth adopting as team policy:

- **Use binary pass/fail, not 1 to 5 scales.** Rating scales produce inconsistent labels and drift toward the middle. If you need finer tracking, measure sub-components separately.

- **One narrow judge per failure mode.** When you use a model to grade outputs (an "LLM-as-judge"), give it one specific failure to check. Validate each judge against human labels, and report how often it catches real failures and how often it correctly passes good outputs, separately, rather than a single "agreement" number.
- **Distrust generic metrics.** "Helpfulness" scores and text-similarity metrics rarely track what users care about. Write evaluators for failures you've actually observed.
- **Name one accountable domain expert.** Someone has to make the final call on what "good" means. On most product teams, that's the PM or a domain expert the PM designates.

Capability, regression and consistency

Anthropic's January 2026 guide "Demystifying evals for AI agents" adds vocabulary that's useful for planning.

Capability evals measure what the product can't yet do well. They should start with a low pass rate: they're the hill you're climbing. **Regression evals** protect what already works, and should pass at nearly 100% on every release. When a capability eval saturates, it graduates into the regression suite.

The guide also distinguishes two ways of counting success over several attempts, and for customer-facing products the difference is large:

- **pass@k** is the probability that at least one of k attempts succeeds. It's the right measure when a user can pick the best of several options.
- **pass^k** is the probability that all k attempts succeed. It's the right measure when a customer expects the product to work every time.

The arithmetic is unforgiving. A task that succeeds 75% of the time on any one try succeeds three times in a row only about 42% of the time (0.75 × 0.75 × 0.75). For an agent that runs the same kind of task hundreds of times a week, consistency, not best-case capability, is what customers experience.

Two more practical rules from the guide: start with **20 to 50 tasks** drawn from real failures and user-reported bugs, balancing "should do" and "should not do" cases; and

grade outcomes, not paths. For an agent, check whether the end state is right (the ticket was updated, the service is healthy), not whether it called tools in exactly the expected order, because agents often find valid routes nobody anticipated.

Are evals the new PRD?

A popular line in 2025 and 2026 holds that "evals are the new PRD." Braintrust's version of the argument is that static specs assume deterministic behavior, while evals define target behavior, acceptance criteria and regression thresholds in a form that can gate releases. Lisa Murkin, writing in Mind the Product, offers the counterpoint: golden datasets overfit, thirty examples are directional rather than proof, datasets drift away from real usage while looking healthy, and a dataset owned only by product recreates the old PRD's problems.

This book's position sits between them. **Evals don't replace the PRD; they become its acceptance criteria in executable form.** The narrative PRD still carries the why, the who, the economics and the risk. The eval suite defines, precisely and testably, what "working" means.

The AI PRD

An AI PRD keeps everything a classic PRD has and adds a set of sections that classic PRDs never needed. Published AI PRD templates, such as Miqdad Jaffer and Paweł Huryn's in *The Product Compass* and Product School's, add model, data and prompt requirements, accuracy and hallucination limits, guardrails and drift monitoring. Jaffer and Huryn's says little about inference cost or unit economics. Here's the full set of differences.

Classic PRD	What the AI PRD adds or changes
Deterministic functional requirements	Behavior specified as **eval sets with pass-rate targets** per failure mode, plus consistency (pass^k) targets
Acceptance by QA test cases	Acceptance by **validated graders** (code checks, calibrated model judges, human review) and regression thresholds in CI
A list of edge cases	A **failure taxonomy** from error analysis, plus adversarial and "should refuse" cases
Performance and security requirements	Adds **latency budgets** (time to first token, end to end), **cost per task**, model and fallback policy, and drift monitoring
Data model	**Data requirements**: grounding sources, permissions, personal data handling, retention and feedback capture
UX flows	**Trust and failure UX**: confidence, citations, correction, undo and escalation
Launch plan	**Staged autonomy**: shadow mode, assisted, supervised, autonomous; kill switch
Pricing as a go-to-market item	**Unit economics as a requirement**: a gross-margin floor per feature
Compliance review at the end	**Regulatory classification up front**: risk tier, transparency duties, documentation obligations

The AI PRD addendum template at the end of this chapter covers every row.

Cost, latency and unit economics

The cost curve is falling, and your bill may still rise

The price of AI capability has fallen fast. Epoch AI's 2025 analysis found that the price of reaching a fixed level of model performance fell by between 9x and 900x per year, depending on the task and threshold. GPT-4-level performance on one widely used knowledge benchmark went from about $37.50 per million tokens in March 2023 to about $0.18 by early 2025. Andreessen Horowitz's Guido Appenzeller called the trend "LLMflation" and put it at roughly 10x per year at constant capability.

But a falling price per token doesn't guarantee a falling cost per task. The steepest cuts apply to models that are no longer the frontier. Reasoning models think in long chains of tokens, and agents call models many times per task, so **tokens per task** tends to rise as products get more capable. Budget and price per task, not per token.

Gross margin is the number to watch

Classic SaaS businesses run gross margins of roughly 75 to 80% or more. AI products run much lower. ICONIQ's 2026 State of AI report projects an average AI-product gross margin of about 53% for 2026, and Kyle Poyar's 2026 monetization survey found a median *target* AI gross margin of about 50%, with only 12% of companies targeting 80% or more. If your margin slides toward those numbers, your finance team will notice before you do.

The levers a PM controls:

- **Model routing**: send easy requests to small, cheap models and hard ones to frontier models. Often the single biggest lever.
- **Prompt caching**: long, stable instructions and context can be cached so repeat calls cost a fraction of the original. Check your provider's current pricing, because it changes often.
- **Workflow caching for agents**: once an agent solves a recurring task, convert the successful path into deterministic code and call the model only when something unexpected happens. Andreessen Horowitz's 2026 analysis of computer-use agents describes this as the pattern that makes agents economical on stable, high-volume work.
- **Budgets and stopping conditions**: maximum steps, tokens, time and spend per task, with an escalation path when a limit is hit.

- **Latency design**: stream output, keep time to first token low on interactive paths, use small models where speed matters, and run long agent tasks asynchronously.

Pricing AI

Seat-based pricing breaks when AI does the work that seats represented. If one agent replaces part of five people's work, charging per seat charges less as you deliver more. That's why AI pricing is in flux.

Model	How it works	Strength	Risk
Seat	Per user per month	Simple, predictable	Revenue falls as AI replaces human effort
Usage or credits	Per token, call, action or credit	Tracks cost	Passes cost volatility to customers; bill anxiety
Outcome	Per resolved ticket, resolved incident or qualified lead	Aligns price with value	Needs a measurable, attributable outcome; vendor carries performance risk
Hybrid	Platform fee plus credits or outcomes	Balances predictability and alignment	More complex to explain and meter

The market is moving toward hybrids. Poyar's 2026 survey of more than 230 B2B software and AI companies found hybrid pricing had become the most common model, at 37% of companies, and 29% using AI credits. The public examples show the experimentation. Intercom's Fin AI agent has charged $0.99 per outcome. Salesforce launched Agentforce at $2 per conversation, then in May 2025 moved to Flex Credits priced at about $0.10 per action, and added per-user options later that year. And Cursor's July 2025 move from request limits to included usage produced surprise charges, a public apology and refunds: a reminder that usage-linked pricing needs metering visibility and spend controls before it launches. Chapter 15 covers AI pricing in depth.

Trust and the UX of being wrong

Calibrated trust

The goal is **calibrated trust**: users trusting the system as much as its reliability warrants, and no more. PAIR's guidebook builds its trust chapters around this idea, along with treating errors and graceful failure as part of the core flow rather than an edge case.

Microsoft's **Guidelines for Human-AI Interaction**, published by Saleema Amershi and colleagues at CHI 2019 and validated with practitioners across twenty products, offer eighteen guidelines grouped into four moments. Paraphrased:

- **Initially**: make clear what the system can do, and how well it can do it.
- **During interaction**: time help to the context, show relevant information, and match social norms while mitigating bias.
- **When it's wrong**: make it easy to invoke, dismiss and correct; scope down when uncertain; and explain why it did what it did.
- **Over time**: learn from behavior cautiously, invite granular feedback, provide global controls, and tell users when the system changes.

Use them as a launch-review checklist. Many AI trust failures map to one of them, often "make clear how well it can do it" or "scope services when in doubt."

Patterns that work

- **Show sources and confidence** wherever the user must act on an output. A root cause with three linked log lines is checkable; a bare assertion isn't.
- **Make "I don't know" a good answer.** Design an explicit low-confidence state, and reward it in evals rather than penalizing it.
- **Make correction cheap** with inline edits, regenerate and undo, and capture every correction as a label for evals.
- **Disclose AI.** Users should know when they're talking to an AI, especially in support. In the EU it's increasingly a legal requirement, too.

- **Degrade gracefully** to search, templates or a human when confidence is low or the model provider is down.
- **Design against rubber-stamping.** "Human in the loop" fails silently when reviewers approve everything. Make wrong output easy to spot: highlight what changed, show the evidence, and sample approvals for audit.

What failure looks like in public

Four widely reported incidents illustrate the failure modes PMs must design against:

- **Air Canada (2024).** A support chatbot told a customer they could claim a bereavement fare retroactively. A Canadian tribunal held the airline responsible for what its chatbot said, rejecting the argument that the bot was a separate entity. Lesson: you own your AI's statements. Ground policy answers in approved sources and show them.
- **Cursor (2025).** An AI support agent invented a login policy that didn't exist, and users cancelled before a co-founder corrected it publicly. Lesson: disclose AI agents, and restrict policy statements to retrieved, approved content.
- **Replit (2025).** During a code freeze, an AI coding agent ran destructive commands against a user's live database, then misreported whether recovery was possible. Replit separated development and production databases afterward. Lesson: least privilege, environment separation and approval for irreversible actions.
- **Deloitte Australia (2025).** A government report produced with AI assistance contained fabricated citations and a fabricated quote; the firm issued a corrected report and refunded part of the fee. Lesson: any AI-assisted deliverable needs citation verification and disclosure.

Data, flywheels and moats

Every AI strategy deck has a data flywheel: more usage produces more data, which makes the product better, which attracts more usage. Many teams assume a flywheel they don't actually have. Usage compounds only when it produces **labels** (signals of

what was good or bad), those labels **feed** evals or model improvements, and users can **see** the improvement.

In B2B, there's a further constraint: enterprise contracts often forbid training on customer data. The flywheel may have to run on per-customer context (runbooks, history, preferences), on your own eval sets, and on aggregate signals customers have consented to share. Write feedback capture into the requirements: what signal, under which consent and contract terms, stored where.

Model access isn't a moat; every competitor can call the same models. The defensible advantages in 2026 are workflow integration and switching costs, proprietary context (the customer-specific knowledge your product accumulates), distribution, and trust. Chapter 16 covers defensibility in depth.

Regulation: a product input now

Regulation used to arrive at the end of an enterprise product cycle, as a legal review. For AI products it needs to arrive at the start, because the evidence regulators require (risk management, logging, human oversight, accuracy testing, documentation) has to be built into the product, not bolted on.

A snapshot as of late 2026, which Chapter 33 covers in depth. This is not legal advice, and dates should be confirmed with counsel before you rely on them.

- **EU AI Act.** The Act entered into force in August 2024. Prohibited practices and AI-literacy duties applied from February 2025, and obligations for general-purpose AI models from August 2025. Transparency obligations, such as telling people they're interacting with AI, apply from August 2026. In July 2026 the EU adopted the "Digital Omnibus" amendment (Regulation (EU) 2026/1744), which postponed the high-risk obligations: stand-alone high-risk systems now from December 2027, and AI embedded in regulated products from August 2028.
- **NIST AI Risk Management Framework.** A voluntary US framework built on four functions (Govern, Map, Measure, Manage), with a 2024 Generative AI Profile

listing risks specific to generative AI, such as confabulation. Widely used in enterprise risk programs.

- **ISO/IEC 42001.** The first certifiable AI management system standard, published in late 2023. It's becoming the AI counterpart of ISO 27001 in enterprise security questionnaires.

For a PM, the practical step is to record in the PRD whether you're a provider or a deployer of each AI system, its likely risk tier, and the transparency and documentation duties that follow. Even where obligations are years away, enterprise customers will ask for this documentation in security reviews now.

How to apply it

Here's the lifecycle for building an AI-powered feature or product, from idea to scale.

1. **Write the cost-of-error table.** For each kind of mistake (false positive, false negative, harmful output, wrong action), write what it costs the user and the business.
2. **Decide whether AI is the right tool.** If a rule-based solution meets the need, use it. If not, state why AI adds value.
3. **Choose the simplest pattern.** Start at the top of the workflow-to-agent table and move down only when a simpler pattern fails. Record the decision.
4. **Set autonomy per action class.** Build the reversibility map. Default anything that changes a system of record to L2 (human approval) until evidence supports more.
5. **Prototype and run error analysis.** Put a prototype in front of real or realistic inputs, read at least 30 traces, and build the failure taxonomy.
6. **Build the eval suite.** Turn the taxonomy into 20 to 50 tasks with binary graders. Separate capability from regression evals. Validate any model judges against human labels. Set pass-rate and pass^k targets with your domain expert.
7. **Build the cost model.** Estimate tokens and model calls per task, cost per task at expected volume, and gross margin at the proposed price. Set a margin floor. Identify routing and caching levers.

8. **Design the trust UX.** Sources, confidence, “I don’t know,” correction, undo, escalation, disclosure. Review against the human-AI interaction guidelines.
9. **Classify the regulatory exposure.** Provider or deployer, risk tier, transparency duties, documentation. Bring counsel in now.
10. **Roll out in stages.** Shadow mode (the AI runs, users don’t see it, you compare to humans); assisted (users see suggestions); supervised (the AI acts with approval); autonomous for allowlisted actions. Move between stages on eval and production evidence, not on the calendar.
11. **Monitor in production.** Track quality on sampled production traffic, cost per task, latency, user corrections and escalations. Re-run regression evals on every model, prompt or provider change.
12. **Close the loop.** Every production failure becomes a new eval case. Review the failure taxonomy monthly.

FOR PRODUCT LEADERS

Make the eval suite, the cost model and the model register required artifacts in your portfolio’s Product 360 for every AI-powered product, and review them in the monthly product review alongside the dashboard. AI products that aren’t measured this way drift in quality and margin without anyone noticing, until a customer or the CFO does.

The AI lens: using AI to build AI products

There’s a recursion worth exploiting here: AI is very good at helping you build the artifacts this chapter describes, as long as you keep the judgment human.

- **Drafting eval cases.** A model can generate dozens of candidate test inputs per failure mode, including adversarial and “should refuse” cases you didn’t think of. You and your domain expert curate them.

- **Clustering error notes.** After you've written free-form notes on traces, a model can propose a first failure taxonomy. Check it against your notes; the model will merge categories that matter.
- **Writing judge prompts.** A model can draft a narrow judge for one failure mode. You must still validate it against human labels. An unvalidated judge is a second model's opinion, not a measurement.
- **Building the cost model.** Give a harness your expected volumes, token estimates and provider prices and it can produce a scenario model in minutes. Check every input yourself.
- **Red-teaming.** A model can play an adversarial user, a prompt injection in a retrieved document, or a confused customer. It finds different weaknesses than your team does.

The guardrail: **never let a model grade its own work unvalidated, and never let it decide what "good" means.** Those are the two decisions that make an eval meaningful.

The Beacon case: the wrong rollback

BEACON CASE **What happened**

Three weeks into the Beacon Agents design-partner pilot, a responder at a retail customer accepts the agent's diagnosis that a deploy to the checkout service caused a latency spike, and rolls it back. The real cause was a database failover in a different region. The rollback takes checkout down for eleven minutes during a sale. The customer's VP of IT Operations calls Northlight's CEO.

The immediate response. You pause the pilot's ability to propose rollbacks, keeping diagnosis on. You write to the customer within 24 hours explaining what happened, what you've changed and what you'll do next. The incident goes in the decision log

and the risk register, and it becomes the first entry in a new AI incident review process (Chapter 33).

Error analysis. You and the applied AI engineers pull 120 pilot traces and read them. Free-form notes cluster into a failure taxonomy of seven modes. The top three:

1. **Recency bias.** The agent over-weights the most recent deploy, even when timing doesn't fit.
2. **Single-region blindness.** The agent doesn't check infrastructure events outside the affected service's region.
3. **Overconfident framing.** Diagnoses are worded with the same certainty whether the evidence is strong or thin.

None of these was in the original PRD, which defined quality as "accurate root cause."

The eval suite. You turn the taxonomy into 48 tasks built from real incidents, half "should diagnose" and half "should say I'm not sure." Each has a binary grader: a code check for whether cited evidence exists and matches the timeline, and a narrow model judge for overconfident framing, validated against 150 labels from two SREs. Recency bias and region blindness become capability evals; the existing behaviors that worked become a regression suite.

The targets. With Priya's team as domain experts, you set the bar: diagnosis accepted by responders in at least 60% of incidents; an "I'm not sure" rate between 15% and 30% (lower means overconfidence, higher means low value); zero rollback proposals where the cited evidence doesn't match the timeline. Because responders see the agent in every incident, you track pass^3 on the regression suite, not just single-run accuracy.

Autonomy and UX. Rollback proposals move from L3 to L2: the agent can propose, but the UI shows the timeline evidence side by side with the change, and the approver must confirm the specific service and region. Every diagnosis now displays its evidence and an explicit confidence band.

Cost. The fix isn't free. Checking cross-region infrastructure events adds three tool calls and about 40% more tokens per diagnosis. Your cost model shows Beacon Agents' gross margin falling from 72% toward 66% at current pricing. You route the

first-pass triage to a smaller model and cache the stable runbook context, which brings the projected margin back to about 71%. The margin floor goes in the AI PRD: 70%.

Pricing. The incident sharpens the per-resolved-incident pricing question. Marcus, the economic buyer, likes paying for outcomes, but his CFO asks who pays when the agent makes an incident worse. You add an outcome definition (incident resolved, with Beacon's diagnosis accepted, and no reopening within 24 hours) and a credit policy for agent-caused incidents to the pricing proposal for Chapter 15.

The result. Six weeks later the pilot restarts with the new evals gating every release. Diagnosis acceptance reaches 64%, the "I'm not sure" rate settles at 22%, and there have been no mismatched rollback proposals. The retail customer stays in the pilot and, at Northlight Live, agrees to be a reference customer for diagnosis, though not yet for autonomous action.

Product 360 artifacts

TEMPLATE **AI PRD addendum**

```
# AI PRD addendum · [Feature] · v[n] · [Date]

Attach to the standard PRD. Every section is required for AI-powered features.

## 1. Why AI
- The user problem (link to PRD):
- Why a rule-based solution isn't enough:
- Cost-of-error table:

| Error type | Example | Cost to user | Cost to business | Detectable by user? |
|---|---|---|---|---|
| False positive | | | | |
| False negative | | | | |
| Harmful or inappropriate output | | | | |
| Wrong action (agents) | | | | |

## 2. Pattern and autonomy
- Pattern: augmented model / chain / routing / parallel / orchestrator / evaluator-optimizer / agent
- Why the simpler patterns weren't enough:
- Autonomy per action class:

| Action class | Reversibility (cheap / costly / irreversible) | Autonomy (L1 to L5) | Approval and confirmation design |
|---|---|---|---|
| | | | |

- Stopping conditions and budgets: max steps [n], max tokens [n], max time [n], max spend per task [$]
- Escalation path when a limit is hit:

## 3. Quality (link to eval plan)
- Failure taxonomy (top modes):
```

```
- Capability eval targets:
- Regression eval threshold (target near 100%):
- Consistency target (pass^k):
- Accountable domain expert:

## 4. Models and data
- Models and providers (link to model register):
- Fallback when the provider is unavailable or degraded:
- Grounding sources and how they're permissioned:
- Personal and customer data handled, retention, residency:
- Training and feedback use permitted by contracts: yes / no / with consent
- Feedback signals captured and where they're stored:

## 5. Cost and latency (link to AI cost model)
- Cost per task at expected volume:
- Gross-margin floor:
- Latency budget: time to first token [ms], end to end [s]
- Cost levers in use: routing / caching / workflow caching / budgets

## 6. Trust and failure UX
- How capability and limits are communicated:
- Sources and confidence display:
- "I don't know" state:
- Correction, undo and regenerate:
- AI disclosure:
- Graceful degradation:
- Anti-rubber-stamping design (for approvals):

## 7. Risk and regulation (link to AI risk assessment)
- Our role: provider / deployer
- Likely risk tier and jurisdictions:
- Transparency and documentation duties:
- Security threats considered: prompt injection, data exfiltration, credential
misuse
```

```
## 8. Rollout
| Stage | Entry criteria (evals and production evidence) | Who sees it | Exit
criteria |
|---|---|---|---|
| Shadow | | | |
| Assisted | | | |
| Supervised | | | |
| Autonomous (allowlisted actions) | | | |

- Kill switch: who can pull it, and how fast it takes effect:
```

TEMPLATE **Eval plan**

```
# Eval plan · [Feature] · [Date]

**Owner:** [PM or domain expert]   **Engineering owner:** [Name]   **Last run:**
[Date]   **Gates releases:** yes / no

## Error analysis
- Traces reviewed: [n] on [dates], from [source]
- Reviewers:
- Failure taxonomy:

| # | Failure mode | Description | Example trace | Frequency in sample | Severity
(H/M/L) |
|---|---|---|---|---|---|
| 1 | | | | | |

## Eval suite
| Suite | Type | # tasks | Grader (code / model judge / human) | Target | Current |
|---|---|---|---|---|---|
| [Failure mode 1] | Capability | | | | |
| [Existing behaviors] | Regression | | | ≥ 98% | |
| Should refuse or say "I'm not sure" | Capability | | | | |
| Consistency | pass^[k] | | | | |

## Judge validation (for each model judge)
| Judge | Failure mode | Human labels | True positive rate | True negative rate |
Last validated |
|---|---|---|---|---|---|
| | | | | | |

## Run policy
- Runs on: every prompt change / model change / provider change / release
- Regression failure policy: block release / require approval from [name]
- Production sampling: [n] traces per [week], reviewed by [name]
```

```
- New failures from production become eval cases within [n] days


## History
| Date | Change | Capability score | Regression score | Notes |
|---|---|---|---|---|
| | | | | |
```

TEMPLATE **AI cost model**

```
# AI cost model · [Feature] · [Date]

**Owner:** [PM]   **Finance partner:** [Name]   **Prices as of:** [Date]

## Per-task cost
| Step | Model | Calls per task | Input tokens | Output tokens | Cached share | Cost per call | Cost per task |
|---|---|---|---|---|---|---|---|
| | | | | | | | |
| **Total** | | | | | | **$** |

Include retrieval, tool and infrastructure costs per task.

## Volume and margin
| Scenario | Tasks per month | Inference cost | Other COGS | Revenue | Gross margin |
|---|---|---|---|---|---|
| Low | | | | | |
| Expected | | | | | |
| High | | | | | |

- Gross-margin floor: [%]
- Cost per task at which the floor is breached: [$]

## Levers
| Lever | Expected saving | Quality risk | Status |
|---|---|---|---|
| Route [step] to a smaller model | | | |
| Cache [stable context] | | | |
| Convert [recurring path] to deterministic code | | | |
| Lower step or token budget | | | |

## Sensitivities
```

```
- If tokens per task rise 50%:
- If the provider raises prices 20%:
- If usage doubles in a quarter:
```

TEMPLATE

Model and vendor register

```
# Model and vendor register · [Product] · [Date]

| Feature | Model and version | Provider | Hosting region | Data sent | Training on our data? | Contract and DPA | Fallback model | Deprecation date | Last eval run | Owner |
|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | |

## Change policy
- Any model, version or provider change requires a regression eval run and sign-off from [name].
- Deprecation notices are reviewed [monthly]; migrations are planned at least [n] months before the date.
- Customers are notified of material model changes per [contract terms].
```

TEMPLATE **AI risk assessment**

```
# AI risk assessment · [Feature] · [Date]

**Owner:** [PM]   **Reviewers:** [security, legal, privacy, responsible AI]   **Next review:** [Date]

## System description
- What it does, for whom, and what decisions or actions it influences:
- Our role: provider / deployer
- Jurisdictions and likely risk classification (confirm with counsel):

## Risks
| # | Risk | Category (accuracy, bias, privacy, security, safety, IP, transparency, over-reliance) | Likelihood | Impact | Mitigations | Residual risk | Owner |
|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |

## Controls
- Human oversight design:
- Logging and audit (what is retained, for how long, who can access it):
- Transparency to users:
- Security: prompt injection, data exfiltration, credential scope:
- Monitoring and incident response (link):

## Documentation customers may request
- [ ] System description and intended use
- [ ] Data handling and residency
- [ ] Evaluation results and known limitations
- [ ] Human oversight and controls
- [ ] Incident response process
- [ ] Relevant certifications or framework alignment (for example ISO/IEC 42001, NIST AI RMF)

## Sign-off
```

```
| Role | Name | Date | Decision |
|---|---|---|---|
| | | | |
```

Prompts

✦ PROMPT **Open-code a batch of AI traces**

```
Goal: Get a fast first pass of error notes on real traces, to speed up (not replace) your own error analysis.
Paste in: 20 to 40 traces (input, output, tool calls, final outcome), with customer data removed or using an approved tool, plus a one-paragraph description of what the feature is supposed to do.

For each trace below, write one or two sentences of free-form notes on anything that went wrong or looks risky: wrong facts, unsupported claims, missed context, bad tool use, overconfident tone, unsafe action, poor format. If nothing went wrong, write "OK." Don't score with a rating scale. Don't group the notes yet.

Then, separately, propose a draft failure taxonomy: 5 to 10 named failure modes, each with a one-line definition and the trace numbers that show it.

[feature description]
[traces]

Check the output: read at least 15 traces yourself before you look at the model's notes, then compare. Expect the model to miss domain-specific failures and to merge modes you'd keep separate. The final taxonomy is yours.
```

✦ PROMPT **Generate eval cases for one failure mode**

```
Goal: Build a balanced set of test cases quickly.
Paste in: the failure mode's name and definition, three real example traces that
show it, and three that show correct behavior.

Generate 30 eval cases for this failure mode in [feature]: [name and definition].
- 15 cases where a correct system should succeed or act, and 15 where it should
refuse, ask for more information, or say it isn't sure.
- Vary difficulty: include obvious cases and subtle ones.
- Include at least 5 adversarial cases (misleading context, conflicting evidence, an
instruction hidden inside retrieved content).
- For each case give: the input, the context the system would have, the expected
outcome (not the exact wording), and a one-line pass/fail criterion.

Base the cases on the real examples; don't invent product capabilities the examples
don't show.

[examples]

Check the output: have your domain expert review every case. Delete any whose
expected outcome you'd argue about; ambiguous cases poison an eval.
```

✦ PROMPT **Draft and validate a model judge**

Goal: Create a narrow LLM-as-judge for one failure mode, plus a plan to validate it. Paste in: the failure mode's definition, 10 labeled examples (5 pass, 5 fail) with the reason for each label.

Write a judge prompt that checks one thing only: [failure mode]. The judge must return PASS or FAIL and a one-sentence reason quoting the part of the output that drove the decision. Include the definition, the labeled examples as few-shot guidance, and instructions to ignore every other quality dimension.

Then write a validation plan: how many human labels I need, how to split them into development and test sets, how to calculate the true positive rate and true negative rate, and what rates I should require before trusting the judge to gate releases.

[definition and examples]

Check the output: run the judge on a held-out set you labeled yourself. Don't use it to gate releases until you've measured both rates on data it hasn't seen.

✦ PROMPT **Build a first AI cost model**

Goal: See the gross-margin effect of an AI feature before it ships.
Paste in: the steps the feature performs, the model for each step, estimated input and output tokens per call, calls per task, current provider prices (with the date), expected monthly task volume and the planned price.

Build a cost model for [feature] with:
1. A per-task cost table by step, including caching assumptions.
2. Monthly cost, revenue and gross margin at low, expected and high volume.
3. The cost per task at which gross margin falls below [floor]%.
4. Three levers to reduce cost (routing, caching, converting recurring paths to deterministic code, budgets), each with an estimated saving and the quality risk it introduces.
5. Sensitivity to tokens per task rising 50% and prices rising 20%.

Show your formulas. Don't invent prices; use only the ones I gave you, and flag any input you had to assume.

[inputs]

Check the output: recompute one scenario by hand. Confirm prices against the provider's current pricing page before sharing the model with finance.

✦ PROMPT **Review an agent's autonomy design**

```
Goal: Find autonomy and permission gaps before a customer does.
Paste in: the agent's list of actions and tools, the permissions each tool has, the proposed autonomy level per action, and the approval UX.

Review this agent design for [product] as a skeptical security reviewer and SRE.
1. For each action, classify reversibility (cheap, costly, irreversible) and say whether the proposed autonomy level (L1 operator to L5 observer) is appropriate. Recommend changes.
2. Identify any tool with broader permissions than its actions need.
3. List ways a prompt injection in retrieved content or tool output could cause a harmful action, and the control that would stop each.
4. Check stopping conditions and budgets. What happens when a limit is hit?
5. Assess whether the approval UX would let a tired reviewer rubber-stamp a wrong action, and suggest design changes.
Finish with the five changes you'd require before production.

[design]

Check the output: walk through the riskiest action end to end with your engineering lead and confirm the model's reading of the permissions is correct.
```

✦ PROMPT **Red-team an AI feature**

Goal: Find failure modes your team hasn't imagined.
Paste in: the feature description, the system instructions or a summary of them, the user types, and the tools and data it can access.

Act as a red team for [feature]. Generate 25 test scenarios across these categories: confused or mistaken users, adversarial users, prompt injection through documents or tool outputs, requests outside the intended scope, high-stakes situations where a wrong answer causes harm, attempts to extract other customers' data, and situations where the right answer is "I don't know." For each, give the scenario, what a safe response looks like and what failure would look like.

Rank the five scenarios you think are most likely to cause real harm in production, and explain why.

[feature details]

Check the output: run the top-ranked scenarios against the real system. Add every failure you find to the eval suite.

✦ PROMPT **Review an AI PRD for gaps**

```
Goal: Make sure an AI PRD covers what classic PRDs miss.
Paste in: the PRD and its AI addendum.

Review this PRD for an AI-powered feature. For each of these areas, say whether it's
covered well, covered weakly or missing, and quote the relevant text: cost of error;
choice of pattern and autonomy; failure taxonomy and eval targets; consistency
targets; model, fallback and data requirements; cost per task and margin floor;
latency budget; trust and failure UX; regulatory classification; staged rollout and
kill switch. Then list the three gaps most likely to cause a production incident or
a margin problem, and the question I should ask the team about each.

[PRD]

Check the output: confirm each "covered well" against the PRD yourself. Bring the
three questions to your next review.
```

Anti-patterns

ANTI-PATTERN **Vibe checks**

The team tries a dozen prompts, the answers look good, and the feature ships. Quality is never measured, so nobody notices when a model update makes it worse. If there’s no eval suite gating releases, there’s no quality bar.

ANTI-PATTERN **Agents for everything**

A multi-step agent is built for a task a single model call with the right context could do. It's slower, costlier and harder to evaluate. Start with the simplest pattern that works.

ANTI-PATTERN **One autonomy level for the whole product**

The agent is either fully supervised (and users find it tedious) or fully autonomous (and someone gets hurt). Set autonomy per action class, based on reversibility.

ANTI-PATTERN **The margin surprise**

The feature succeeds, usage grows, and finance discovers gross margin has fallen ten points. Cost per task and a margin floor belong in the PRD, and on the monthly dashboard.

ANTI-PATTERN **Unvalidated judges**

A model grades the outputs, the scores look great, and nobody has checked whether the judge agrees with humans. An unvalidated judge measures nothing.

ANTI-PATTERN **Human in the loop, in name only**

Every action requires approval, reviewers approve 99.8% of them in under two seconds, and the one wrong action sails through. Design approval screens that make wrong output easy to see, and audit a sample of approvals.

ANTI-PATTERN **Regulation at the end**

The product is built, and then legal asks for logging, documentation and human-oversight evidence it doesn't have. Classify regulatory exposure at the start, and build the evidence in.

Checklist

- [ ] A cost-of-error table exists, and it drives eval targets, autonomy and UX.
- [ ] The PRD explains why AI is the right tool, and why simpler patterns weren't enough.
- [ ] Autonomy is set per action class, with a reversibility map and approval design for anything costly or irreversible.
- [ ] Stopping conditions and budgets (steps, tokens, time, spend) are defined, with an escalation path.
- [ ] Error analysis on at least 30 real traces produced a failure taxonomy.
- [ ] The eval suite has capability and regression sets, binary graders, validated judges and consistency (pass^k) targets, and it gates releases.
- [ ] A named domain expert owns the definition of "good."
- [ ] The AI cost model shows cost per task, gross margin at three volumes and a margin floor.
- [ ] The model and vendor register lists every model, version, provider, data flow, fallback and deprecation date.
- [ ] The trust UX covers sources, confidence, "I don't know," correction, disclosure and graceful degradation, and it was reviewed against the human-AI interaction guidelines.
- [ ] Regulatory role, risk tier and documentation duties are recorded, and counsel has reviewed them.

- [ ] The rollout is staged (shadow, assisted, supervised, autonomous) with evidence-based entry and exit criteria and a kill switch.
- [ ] Production monitoring covers quality, cost, latency, corrections and escalations, and production failures become eval cases.
- [ ] Any MCP server or API that agents use to reach the product has an owner, scope, authentication, versioning and pricing position.
- [ ] The eval plan, cost model, model register and risk assessment are linked from the Product 360 index.

Key takeaways

- AI products behave like distributions, cost money per use and earn trust one answer at a time. Specifying, measuring, pricing and running them works differently than it does for classic software.
- Start with the cost of being wrong. It sets your eval targets, autonomy levels and UX, and often your pricing.
- Use the simplest pattern that works. Reserve agents for open-ended tasks, and set autonomy per action class based on how reversible each action is.
- Evals are the PRD's acceptance criteria in executable form. Build them from error analysis on real traces, use binary graders and validated judges, and track consistency, not just best-case accuracy.
- Cost per task and gross margin are product requirements. Falling token prices don't guarantee falling task costs, and AI margins run well below classic SaaS.
- Design for calibrated trust: show evidence, make "I don't know" a good answer, make correction cheap and stop human review from becoming a rubber stamp.
- Regulation is now a product input. Record your role, risk tier and obligations at the start, and build the evidence into the product.